

コンピュータ アーキテクチャ I 第4回

塩谷 亮太 (shioya@ci.i.u-tokyo.ac.jp)

東京大学大学院情報理工学系研究科 創造情報学専攻

課題3の解説

課題 3

- 以下では 2進数は 0b... 16進数は 0x... と表記するものとする
- 以下の 16進数を 2進数で表記せよ
 - (1) 0x1011
 - 0b0001 0000 0001 0001
 - (2) 0x1234abcd
 - 0b 0001 0010 0011 0100 1010 1011 1100 1101
 - 2進数を 4桁に区切って変換していくと良い

課題 3

■ 以下の2進数を16進数で表記せよ

- (3) 0b10 1011
 - 0x2b
- (4) 0b1111 0000 1111
 - 0xf0f

■ 以下の演算を行え

- (5) 0b11011010 / 0b10 (「/」は割り算)
 - 0b110 1101
 - 10進に変換して普通に割り算して戻す等してもよいが,
0b10 は 2 なので, / 2 は 2 進数だと右に一桁ずらす事に相当

前回の振り返り

2進数や16進数による数値表現

- 現代のコンピュータは基本的に2進数ベースで出来ている
 - 電圧が「高い=1」 or 「低い=0」のような形で表現
 - 2進数だと回路を作るのが簡単
 - 中間の電圧とかを扱うのは大変
- 2進の利点と欠点
 - 利点：各桁に意味を持たす事が多いので色々と考えやすい
 - 欠点：桁数が大きく人間に把握が難しい
- 16進の利点
 - 利点：2進数とは相互変換が容易
 - 欠点：慣れていないとやや把握が難しい

実際の命令セットの例

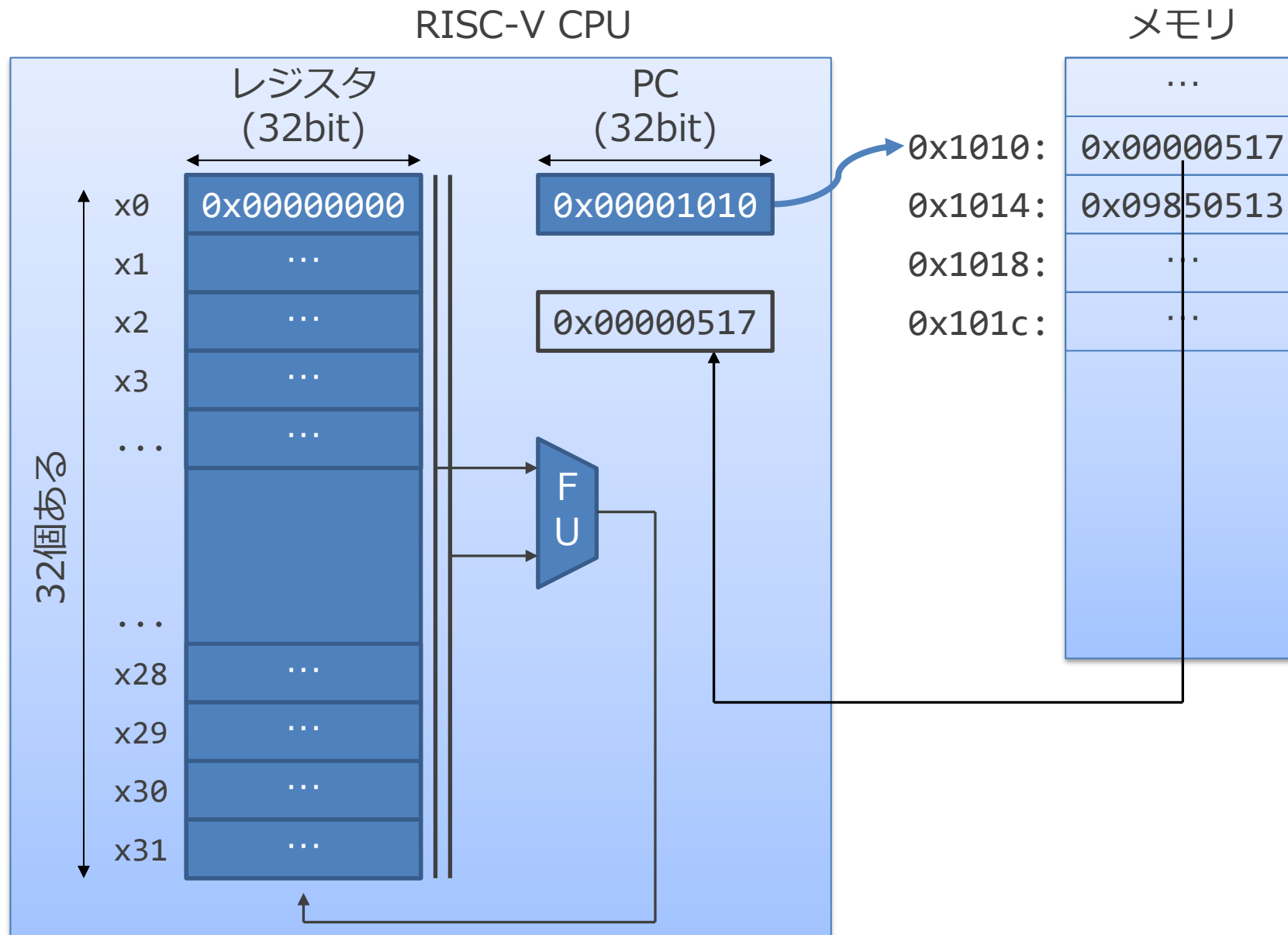
■ RISC-V を例として説明

- 加減算, 論理演算, ロード・ストア, 即値, 分岐とジャンプなど
- 各命令は基本的に 32bit 幅

■ 1 回目の講義で 4 桁の数字で表していたことと同じことを 32bit の中を細かく区切ってやっている

31	25 24	20 19	15 14	12 11	7 6	0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[31:12]					rd	opcode	U-type

RISC-V (32bit) のイメージ



組み合わせ回路/順序回路

1. 組み合わせ回路

- 出力が、現在の入力のみにより決定される論理回路

2. 順序回路

- 出力が、過去の入力（の履歴）にも依存する論理回路

■ 任意の組み合わせ回路/順序回路は、

- 原理的には、完全集合の要素の組み合わせに落とし込める
- たとえば {AND, OR, NOT} を組み合わせれば、全部つくれる

論理回路の実現方法

論理回路の作り方

■ 前回の内容：

- AND, OR, NOT があれば, どんな論理回路も作れる
 - 組み合わせ回路も順序回路も
- CPU を構成する演算器や PC, レジスタ, メモリも作れる
 - 具体的な作り方の例をそれぞれ説明

論理回路の作り方

■ 今回の内容：

- ゲート回路：AND や OR のような論理関数を実現した回路
 - この名前は門を開けたり閉めたりすることから来ているはず
 - 略してゲートとも言う
- ゲート回路はどのように作るのか？
- それらのゲート回路の「遅延」や「消費電力」とは何なのか？
 - （なぜ物理的実体のない情報を扱っているのに時間や電力がかかるのか？

論理回路の作り方

- 以下のそれぞれの仕組みを使った回路を説明
 - リレー
 - CMOS
- これらの論理的な動作は実はほぼ同じに作れる
 - リレーの方が直感的にわかりやすいのでこちらから説明
- 注意：
 - この資料で説明する, N型/P型 を使ったやり方は後の CMOS の説明をわかりやすくするためのものです
 - 実際に用いられていたリレー計算機ではもう少し効率の良い違う方法が使われていたと思う

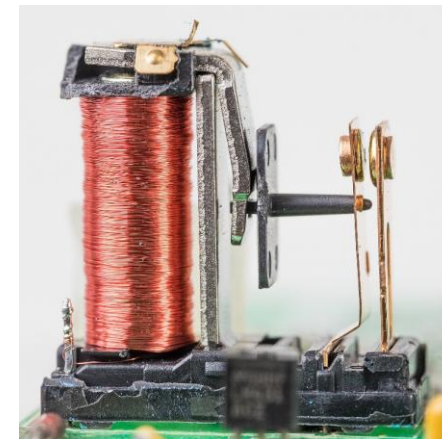
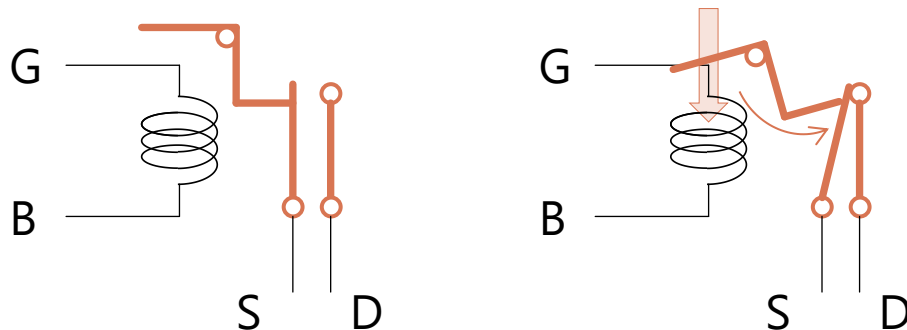
リレー (Relay)

- 電氣的に動作するスイッチ

- もともとは長距離の電信の中継のために作られた…らしい

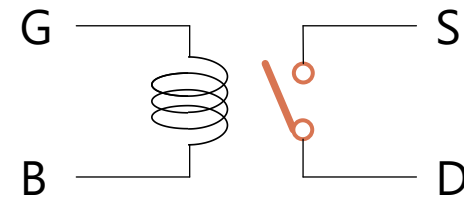
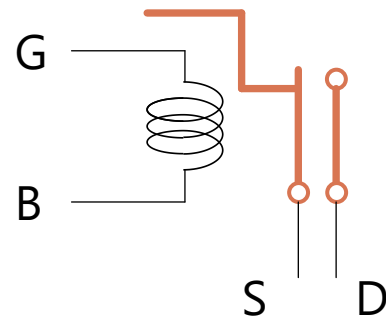
- 以下の図のものの場合,

1. 普段は電極部分がバネになっていて OFF になっている
2. G~B 間に電圧をかけると,
3. コイルが磁石となって, 上の金属を引き寄せ,
4. S に繋がっている端子が右に押し出されて,
5. S と D が接続されて ON に



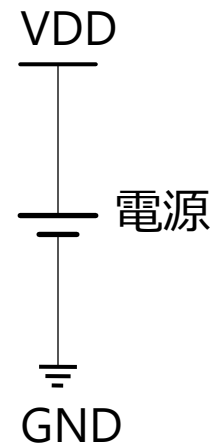
リレーの絵を回路記号に単純化

- わかりにくいので、以降では右のように単純化して表記
 - G～B 間に電圧をかけると、S～D が ON に



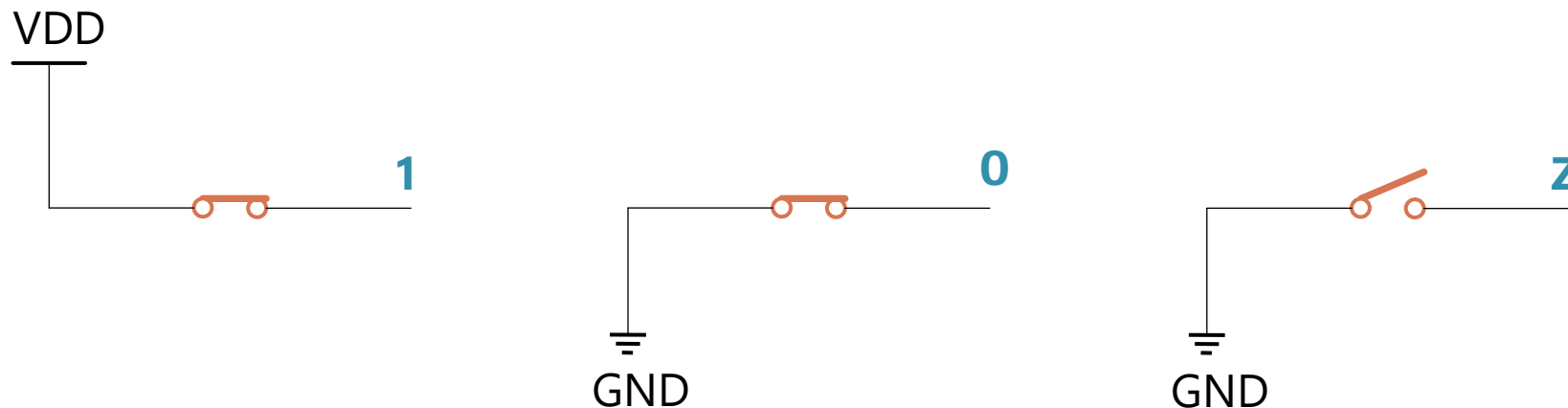
VDD と GND

- 以下の2端子を定義
 - VDD : 高電位
 - GND (Ground=地面) : 低電位
- 記号の描き方
 - 基本的に絵の上側に VDD, 下側に GND を描く
 - VDD と GND があったら, 図内に描かれていない世界のどこかで以下のように電源に繋がっていると考え



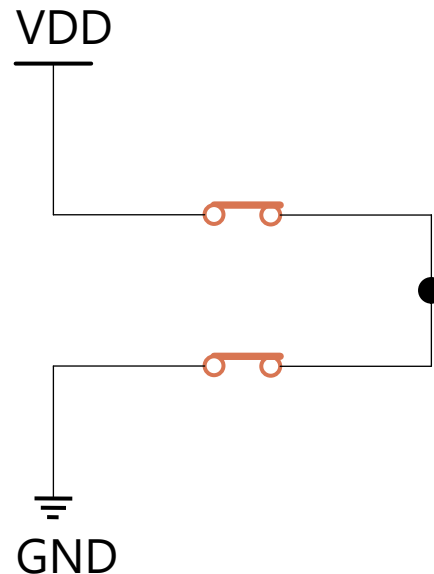
電位と数値表現

- 高電位を 1 , 低電位を 0 , 解放を Z と定義する
- 以下のように読み替えてもよい
 - VDD に直結している信号線の電位 (=状態) は 1
 - GND に直結している信号線の電位 (=状態) は 0
 - どこにも繋がっていない信号線の電位 (=状態) は Z



短絡（ショート）は起きない想定

- 下記図のようにVDD と GND の双方に同時に直結することはない
 - 実際にやると VDD から GND に大電流が流れて回路が壊れる
 - なので、そういう状態を絶対作らないように回路を組む
- もしやってしまった場合、以下の黒点は VDD と GND の中間ぐらいの電位になっていると思う

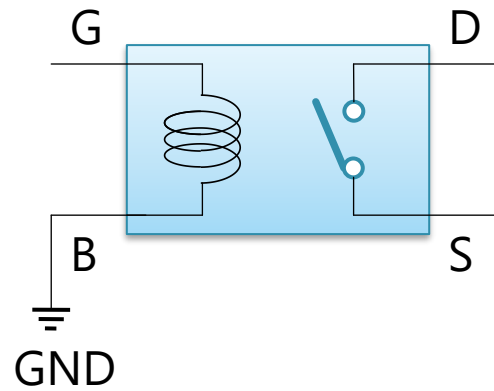


リレーを使って論理回路を作る

- 以下の2つの繋ぎ方をしたリレーを使って論理ゲートを作る
 - N 型
 - P 型

N 型リレー

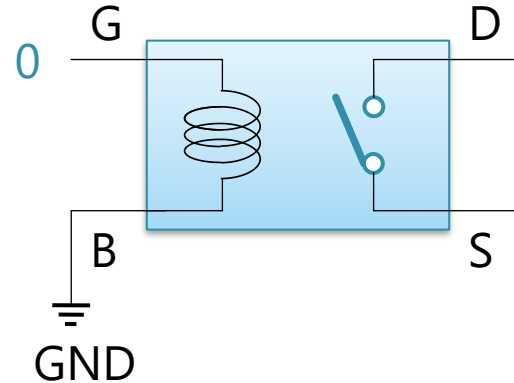
- 左図のように B を GND（低電位）に接続する
 - N は Negative の略



N 型の動作

■ G が低電位 (= 0) → スイッチ OFF

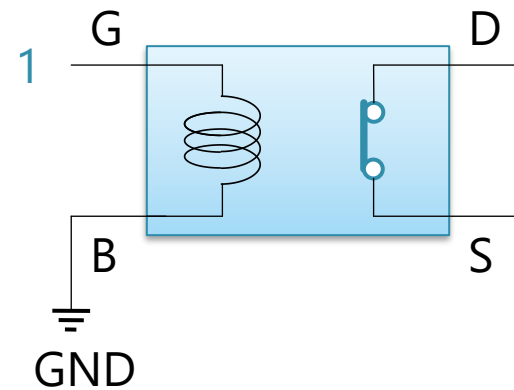
- G~B 間は電位差がないのでコイルが働かない



G	D~S
0	OFF
1	ON

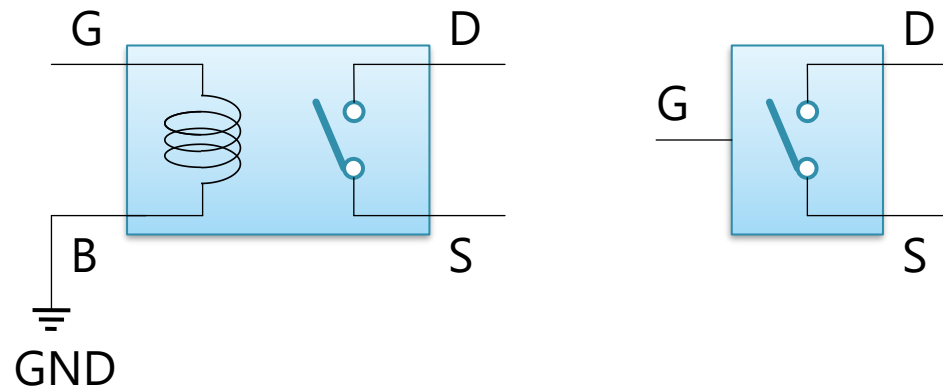
■ G が高電位 (= 1) → スイッチ ON

- G~B 間は電位差がありコイルが動作



N 型リレー

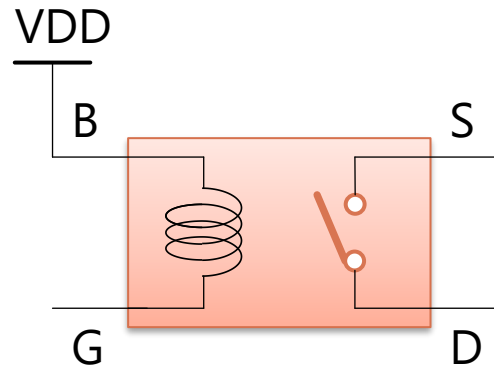
- 左の省略系として右のように描く
 - 毎回ぜんぶの場所にコイルや GND を描くとややこしい



G	D~S
0	OFF
1	ON

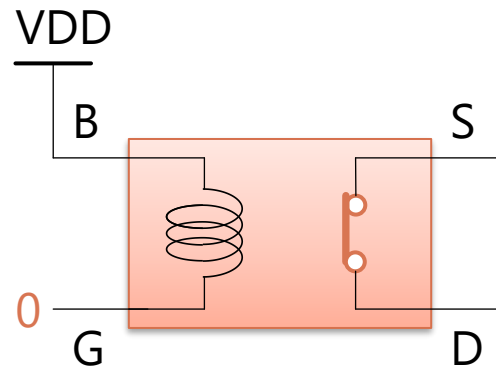
P 型リレー

- 以下のように B を **VDD（高電位）** に接続する
 - G が入力, D が出力
 - P は Positive の略
- 注意
 - N 型の時とは B と G, S と D が上下逆になっているのに注意
 - N 型の場合は B が GND（低電位）に繋がっていた



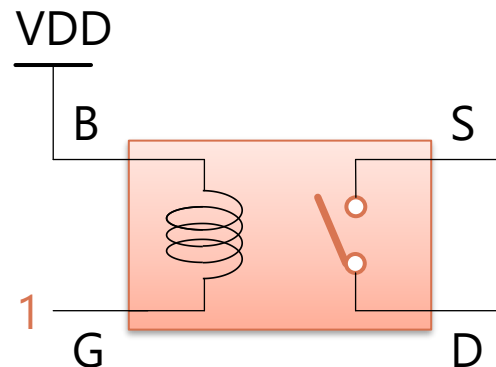
P 型の動作

- G が低電位 (= 0) → スイッチ ON
 - B~G 間に電位差があるのでコイルが動作



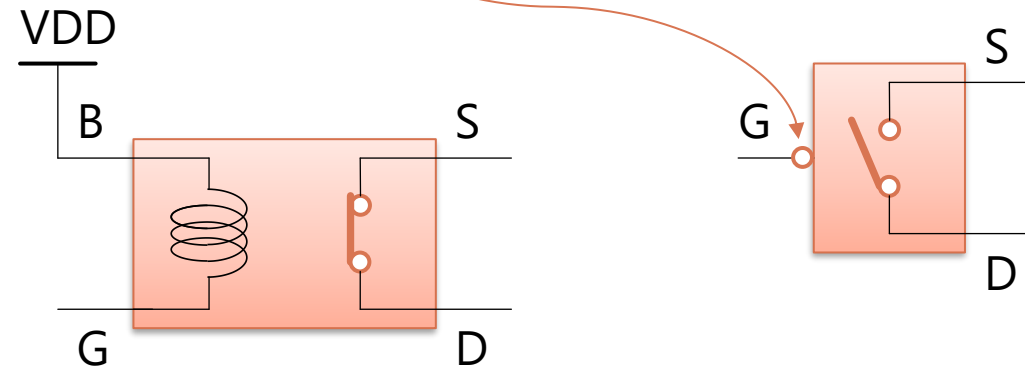
G	D~S
0	ON
1	OFF

- G が高電位 (= 1) → スイッチ OFF
 - B~G 間に電位差がないのでコイルが動作せず



P 型リレー

- 左の省略系として右のように描く
 - G のところに白丸があるのが N 型との描き方の違い

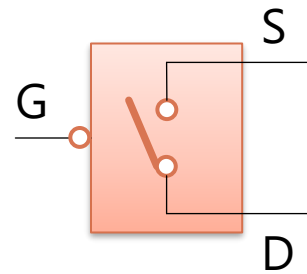
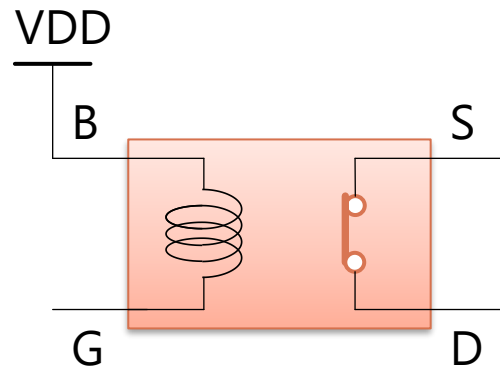


G	D~S
0	ON
1	OFF

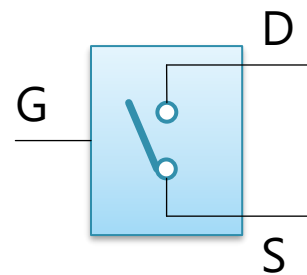
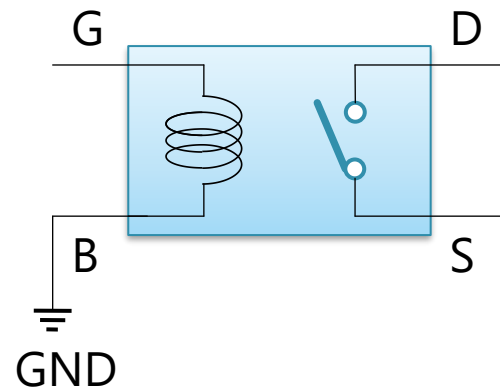
P 型と N 型のまとめ

■ P 型と N 型の違い：

- P 型では G が 0 の時 ON, N 型は G が 1 のとき ON で逆
- G に白丸があるのが P 型, ないのが N 型



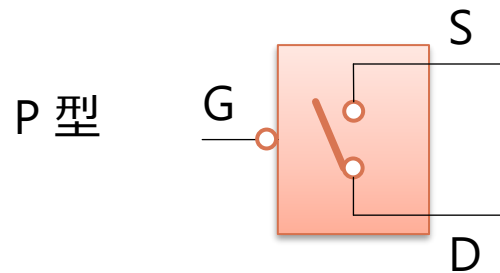
G	D~S
0	ON
1	OFF



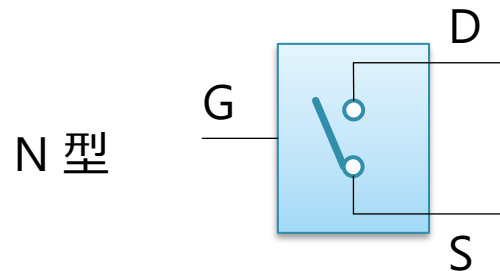
G	D~S
0	OFF
1	ON

P と N が, どっちかどっちだかわからない?

- 論理回路では白丸は反転を表す
 - 白丸なしの N型は 1 で ON , 0 で OFF
 - 白丸があったらそこで反転するので 0 で ON
...と憶えると良いかもしれない



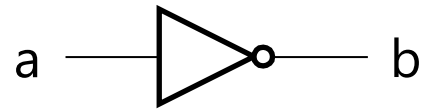
G	D~S
0	ON
1	OFF



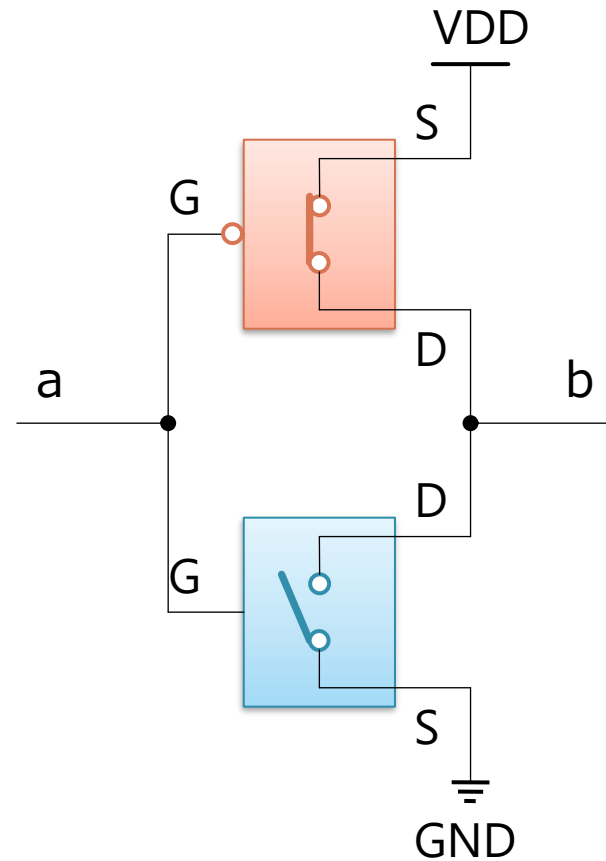
G	D~S
0	OFF
1	ON

NOT ゲート

- P 型と N 型を並べて右のように繋ぐと NOT ゲートができる



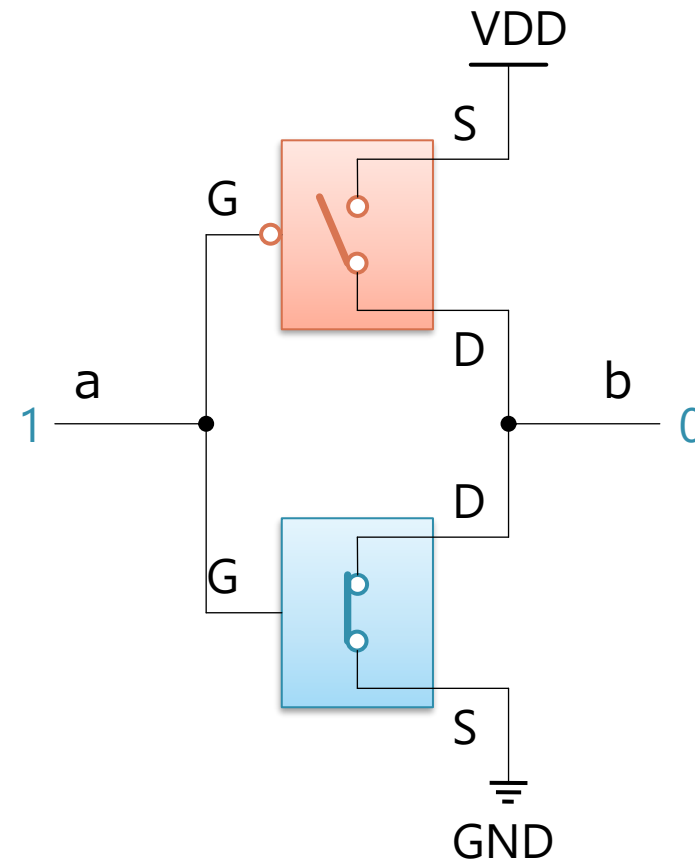
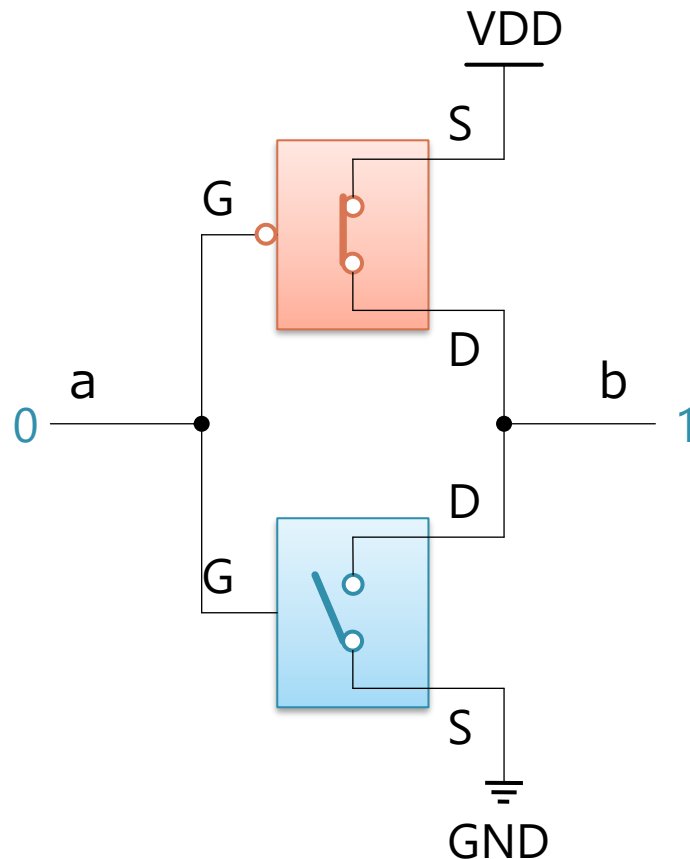
a	b
0	1
1	0



NOT ゲートの動作

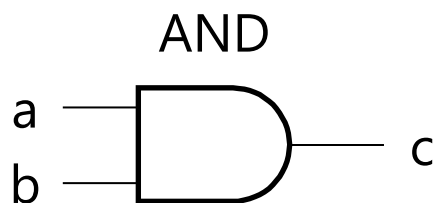
- 双方のスイッチが必ず反対の動作になる
 - P 型は入力が 0 だと ON
 - N 型は入力が 1 だと ON

a	b
0	1
1	0

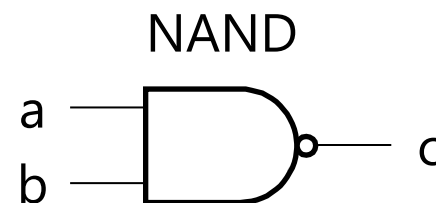


NAND ゲート

- NAND は AND の結果の NOT をとったもの
 - 左側が AND 右側が NAND
 - NAND は AND の後ろに白丸（反転）がある形で描く



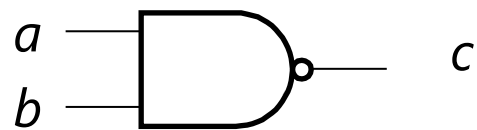
a	b	c
0	0	0
0	1	0
1	0	0
1	1	1



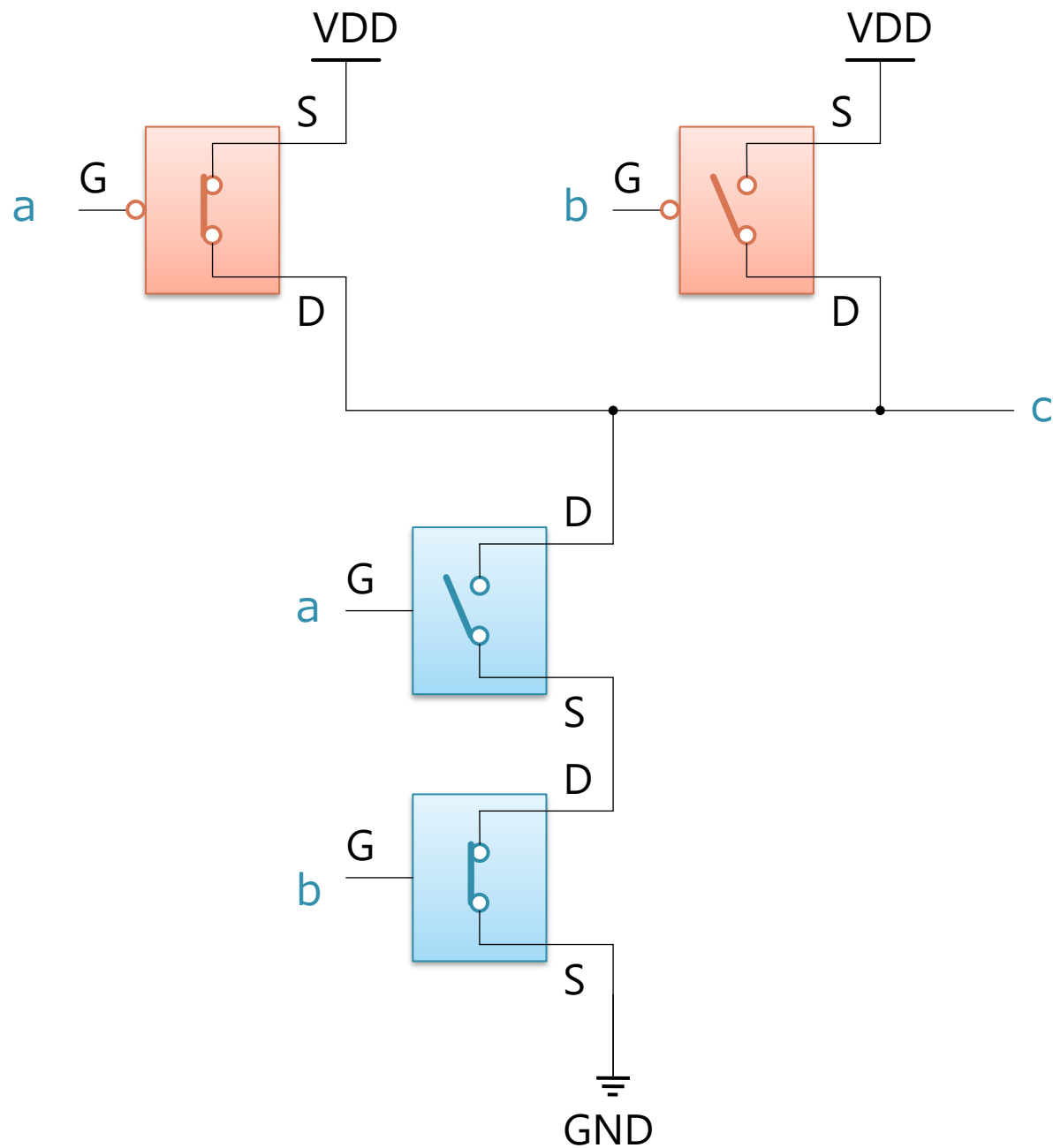
a	b	c
0	0	1
0	1	1
1	0	1
1	1	0

NAND ゲート

- 右のように接続すると NAND ゲートが作れる



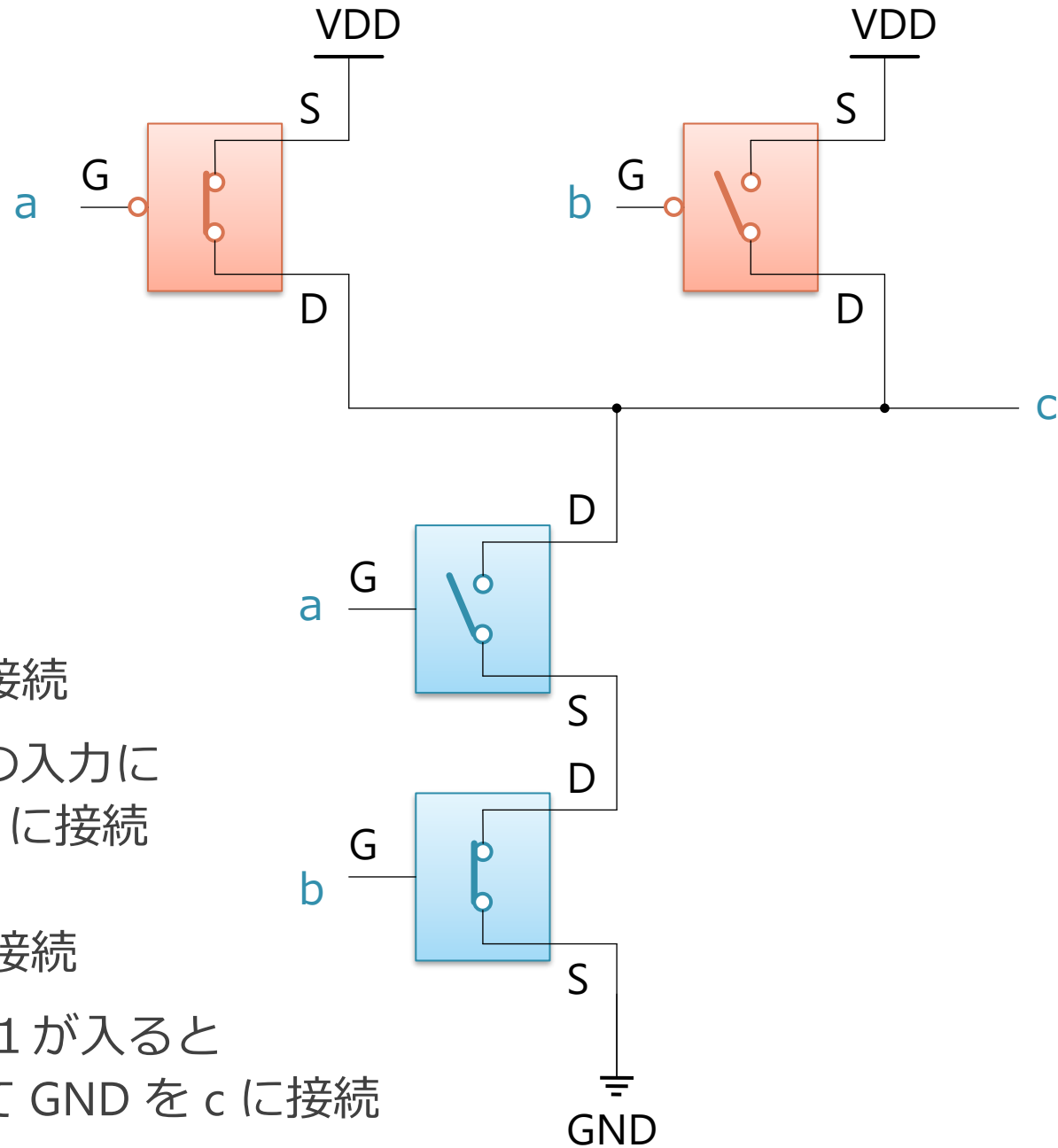
a	b	c
0	0	1
0	1	1
1	0	1
1	1	0



NAND ゲートの動作

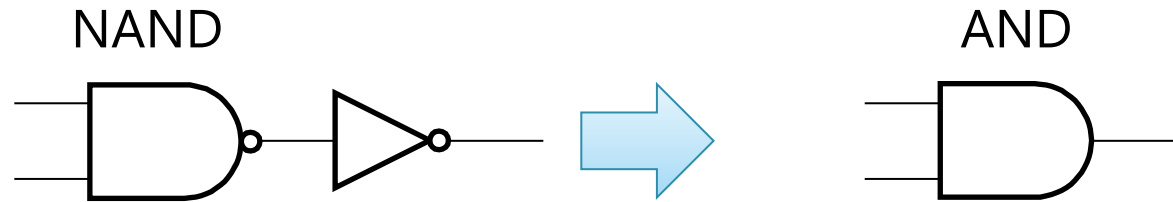
a	b	c
0	0	1
0	1	1
1	0	1
1	1	0

- 上側の P 型 2 つは並列接続
 - a と b どちらか片方の入りに 0 が入ると VDD を c に接続
- 下側の N 型 2 つは直列接続
 - a と b 双方の入りに 1 が入ると 2 つとも ON になって GND を c に接続



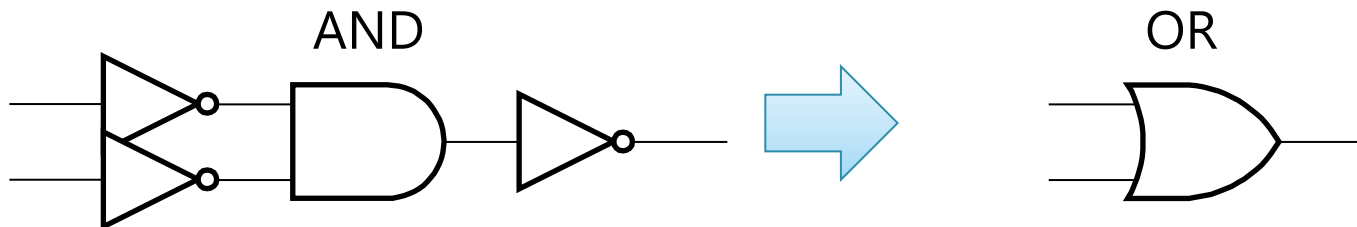
NOT と NAND から AND と OR を作る

- NAND の出力に NOT をつけると AND が作れる



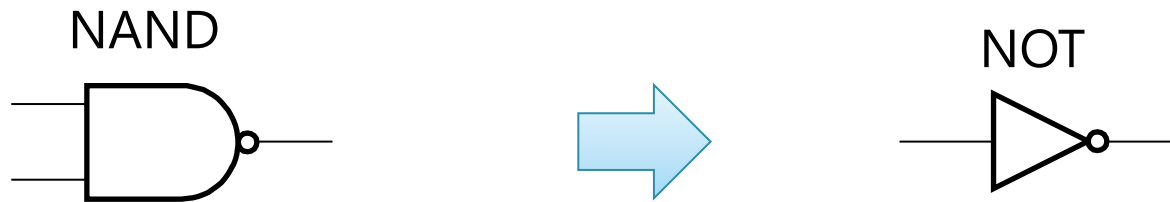
- AND の入力と出力に NOT をつけると OR が作れる

- ド・モルガンの定理 : $\text{NOT}(a \text{ OR } b) = \text{NOT}(a) \text{ AND } \text{NOT}(b)$
- 両辺に NOT を適用 : $a \text{ OR } b = \text{NOT}(\text{NOT}(a) \text{ AND } \text{NOT}(b))$



NAND が作ればそれだけでも良い

- NAND の2つの入力に同じものを入れれば NOT になる



リレーからコンピュータを作る

- リレーを使って NOT, AND, OR を作る
 1. リレーを使って P 型と N 型 のリレーを作る
 2. P 型と N 型を組み合わせで NOT と NAND を作る
 3. NOT と NAND を組み合わせで AND と OR を作る

リレーからコンピュータを作る

- リレーを使って NOT, AND, OR を作る手順
 1. リレーを使って P 型と N 型 のリレーを作る
 2. P 型と N 型を組み合わせて NOT と NAND を作る
 3. NOT と NAND を組み合わせて AND と OR を作る

- コンピュータを作る手順（前回の講義）
 1. NOT, AND, OR を使うと任意の論理関数と記憶回路が作れる
 - =任意の組み合わせ回路と順序回路が作れる
 2. 任意の組み合わせ回路と順序回路を使うと CPU の部品が作れる
 - PC, レジスタ, 演算器, メモリ...
 3. それらを組み合わせるとコンピュータができる

リレーがあればプログラムが実行できる

- コンピュータがあれば、プログラムが実行できる
 1. コンピュータは機械語を解釈して実行できる
 2. 機械語はアセンブリ言語から変換できる
 3. アセンブリ言語はC言語からコンパイルできる
- 途中に多数のステップがあるが、まとめると,
 - リレーがあれば, C言語のプログラムを実行できる

つまり,

- なんらかの入力に従って ON/OFF できるスイッチがあれば,
それでプログラムが実行できるコンピュータが作れる
 - 具体的な回路の組み方は, 回路の種別ごとに違う事もある
 - NAND さえ出来ればこっちのもの

余談：いろいろなコンピュータ

- なんらかの入力に従って ON/OFF できるスイッチさえあれば, プログラムが実行できるコンピュータが作れる
- (余談) いろいろなコンピュータ
 - リレー式
 - 真空管
 - レッドストーン回路

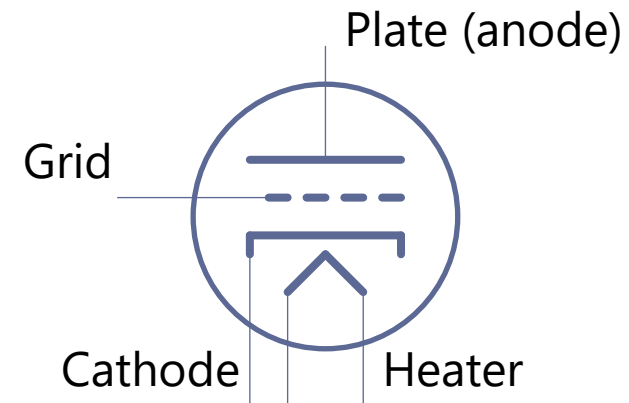
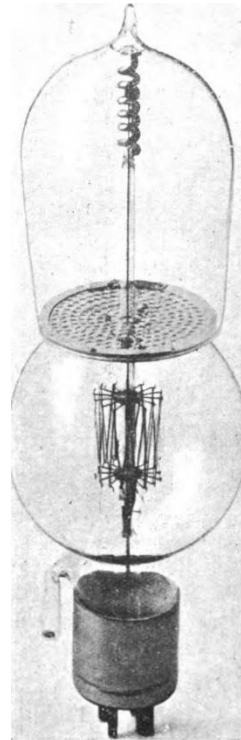
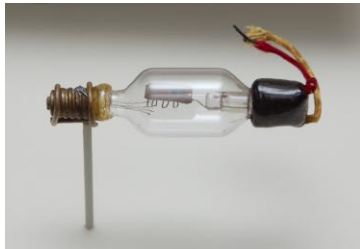
リレー式計算機 富士通 FACOM128B (1956)



画像は <https://pr.fujitsu.com/jp/news/2018/08/21.html> より

真空管（三極管）

- 電球にカソードとグリッド, プレートなどの電極を追加したもの



画像は <https://en.wikipedia.org/wiki/Triode> より
De Forest Audion tube と Lieben-Reisz tube
1900 年過ぎの発明

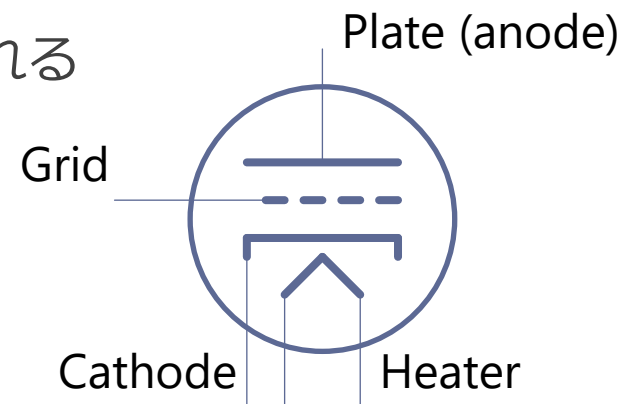
真空管の動作

■ 動作：

1. Plate に正の電圧を，Cathode に負の電圧をかける
2. Heater（電球のフィラメント）に電源を接続
3. 暖められた Cathode が電子を放出
4. Plate に電子が吸い寄せられる
 - 電子が移動する = それらの間に電流が流れる

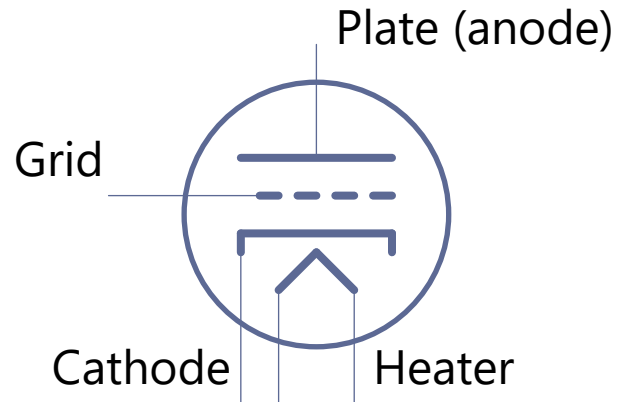
■ スイッチング

- なにもしなければ Grid は網なので電子はすき間を通過
- Grid に負の電圧をかける
- 負同士ではじきあうので，電子が上にいけなくなり電流が OFF

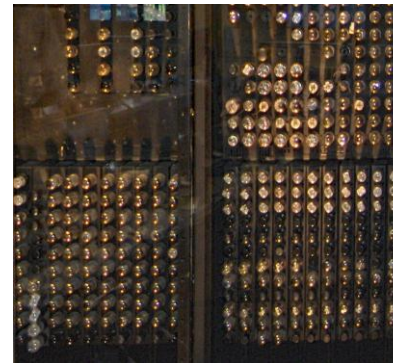
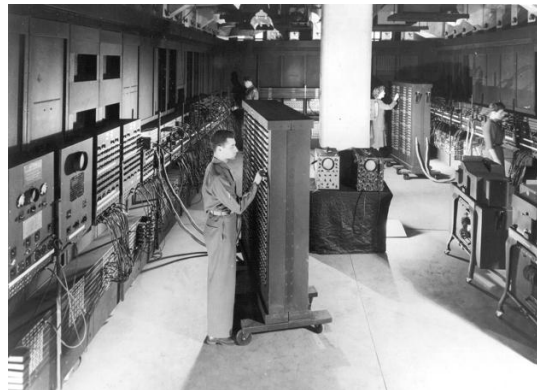


真空管

- Grid にかける電圧で, Cathode と Plate を ON/OFF できる
 - つまり, これでもコンピュータが作れる



- ENIAC (1945)
 - 世界最初のプログラムが実行できるコンピュータ
 - 18,000 個の真空管で出来ている



レッドストーン回路

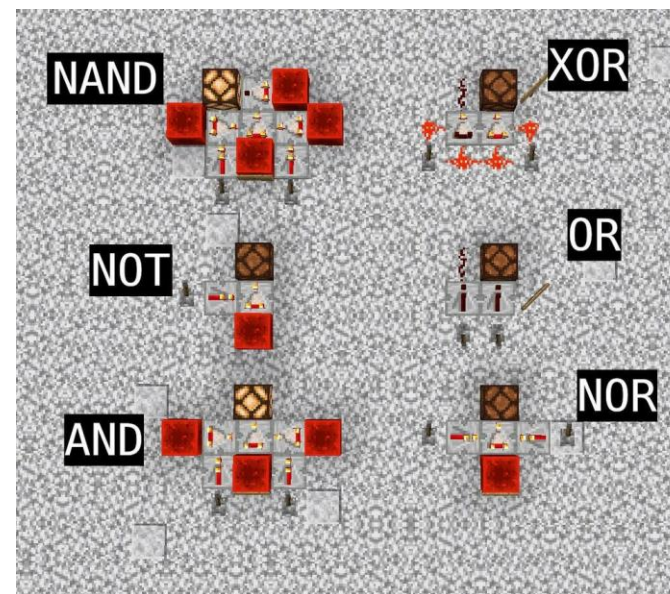
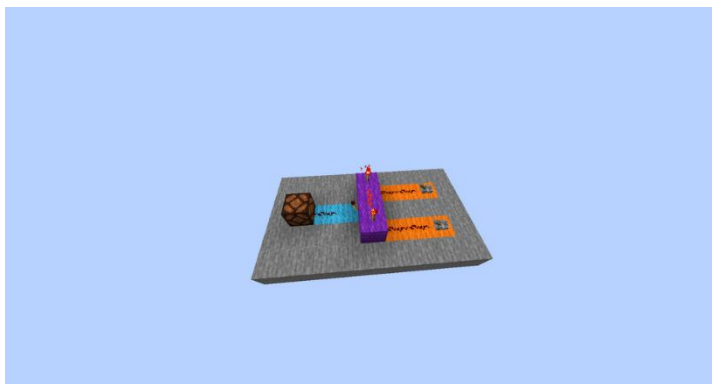
- Minecraft というゲームの中で作られている回路
 - <https://www.minecraft.net/ja-jp>
 - もともとは箱庭でブロックをおいて色々つくって遊ぶゲーム



レッドストーン回路

- 「レッドストーン」というゲーム内の要素を利用
 - 有効と無効の状態を持つ
 - スイッチと扉を繋げて、遠くから開けたり閉じたりとかする
- これも入力で出力を制御出来ていることに誰かが気づいた

AND ゲート
右のスイッチを2つとも入れると
左側が赤くなる



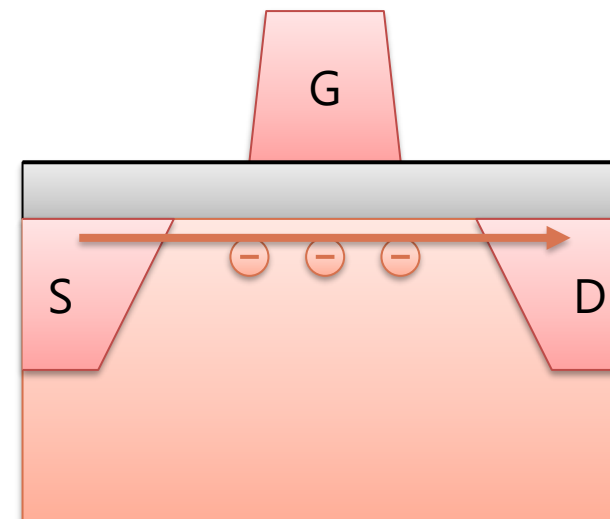
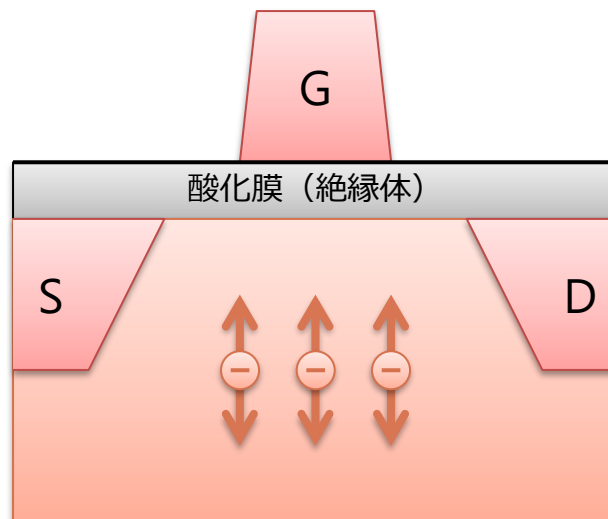
CMOS

論理回路の作り方

- 以下のそれぞれの仕組みを使った回路を説明
 1. リレー
 2. CMOS

MOS FET

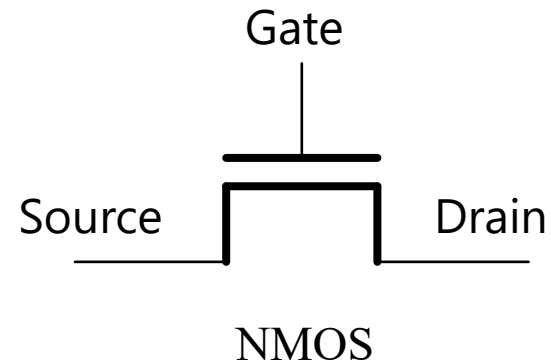
- MOS: metal-oxide-semiconductor
 - 上から, 金属, 酸化膜, 半導体のサンドイッチ構造
 - 酸化膜を挟んで平行板コンデンサが構成されている
- 電界効果トランジスタ (FET: Field-Effect Transistor)
 - 電界で電子を動かしてスイッチングする
 1. G に電圧をかけてコンデンサに充電
 2. 電子の層 (チャネル) 酸化膜下にできて, S と D が繋がり ON に



MOS には NMOS と PMOS の2つがある

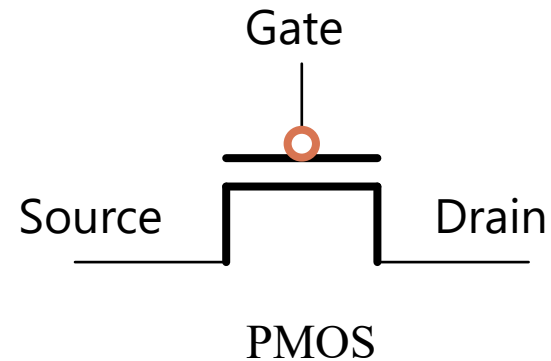
■ Negative (N)

- 電子がキャリア
- 低電位しか通せない
- 接地側に配置



■ Positive (P)

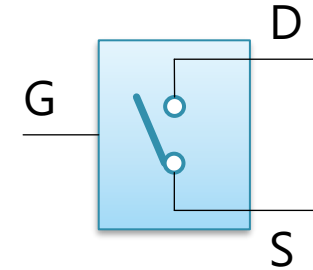
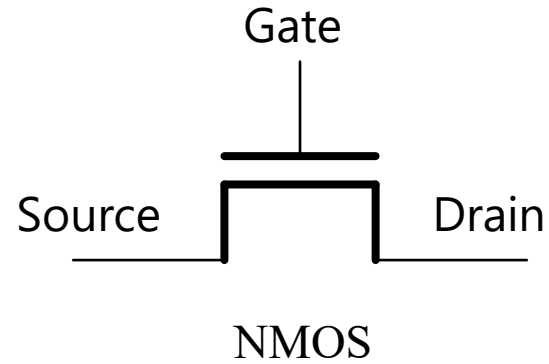
- 正孔 (hole) がキャリア
- 高電位しか通せない
- 電源側に配置
- Gate に○がついている



N 型/P 型リレーとほぼ同じ動作

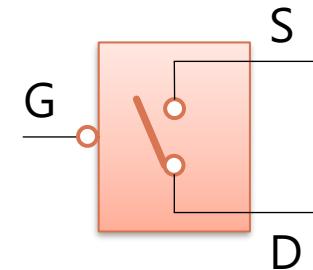
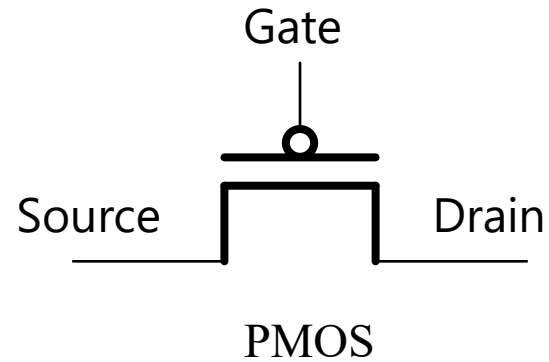
■ Negative

- Gate を 1 にすると ON になる



■ Positive

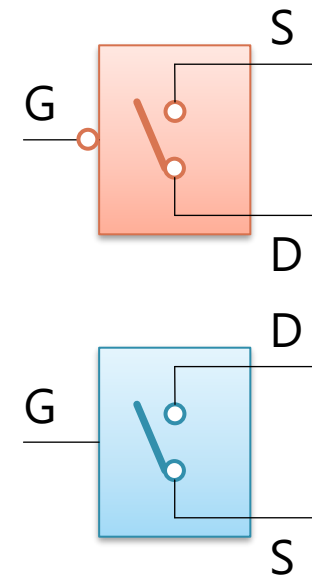
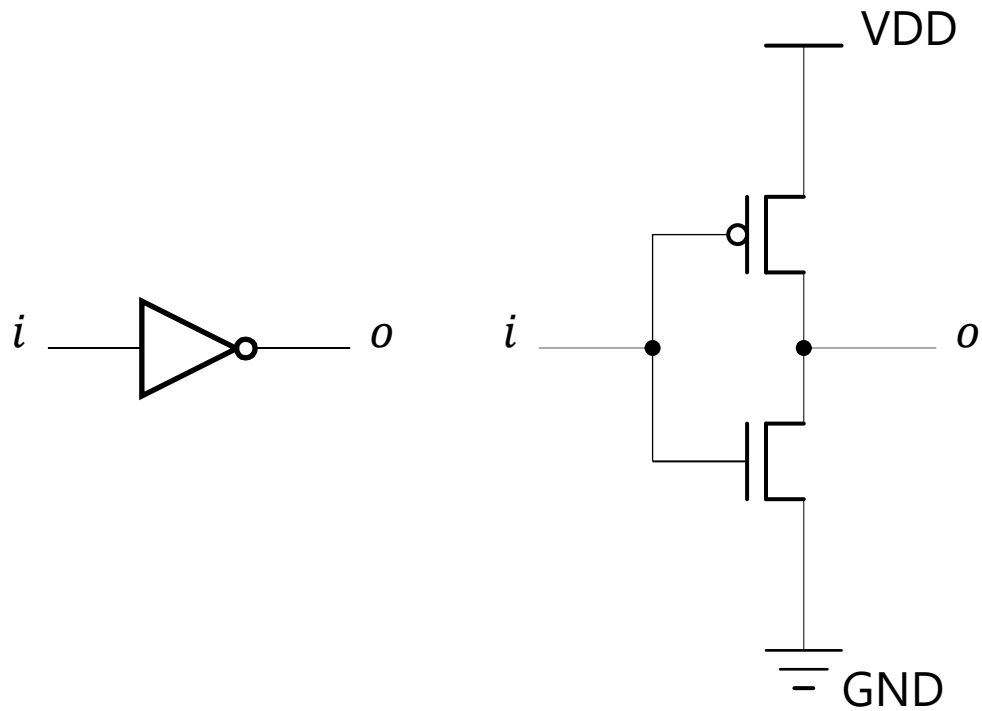
- Gate を 0 にすると ON になる



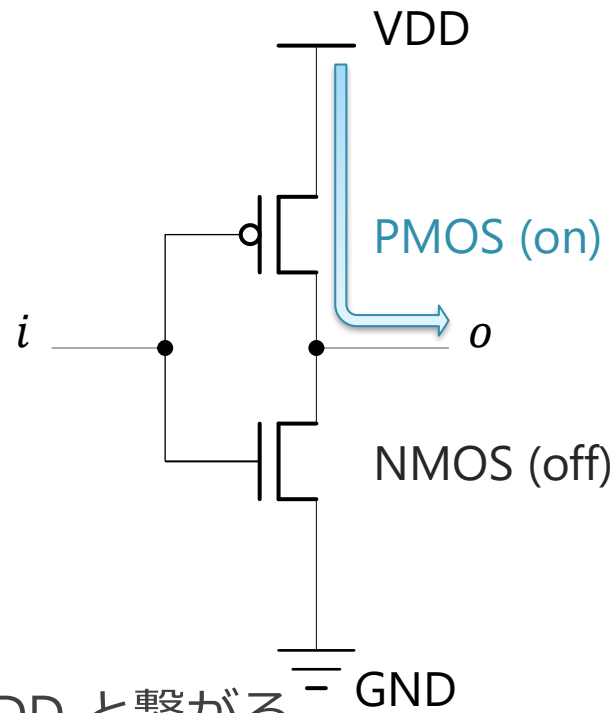
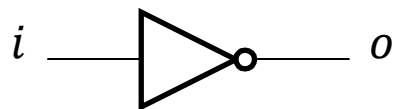
CMOS: Complementary Metal–Oxide–Semiconductor

- CMOS は NMOS と PMOS の 2 種類のトランジスタから成る
 - 電界で電荷（電子/正孔）を動かして ON/OFF する
- 基本的に N 型/P 型リレーとほぼ同じ動作
 - つまり全く同じように論理回路が組める
- 現代のコンピュータはこの CMOS を使って作られている

NOT ゲートの例



NOT ゲートの例

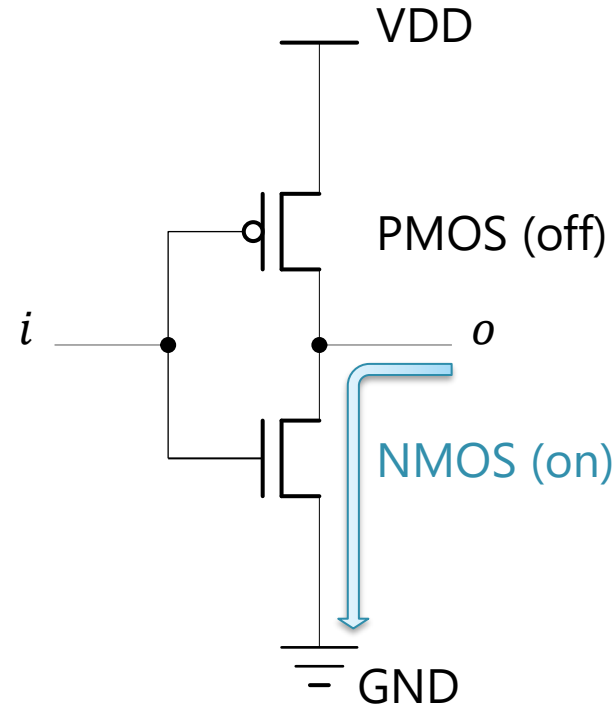
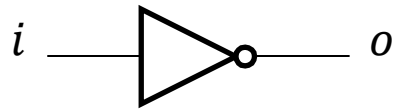


■ i が 0 (低電位) の場合

- PMOS (上側) は on = VDD と繋がる
- NMOS (下側) は off

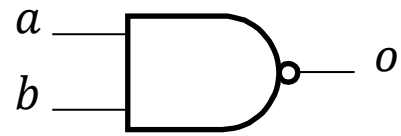
■ 出力が 1 (高電位に)

NOT ゲートの例



- i が 1 (高電位) の場合
 - PMOS (上側) は, off
 - NMOS (下側) は, on = GND と繋がる
- 出力が 0 (低電位に)

NAND の例



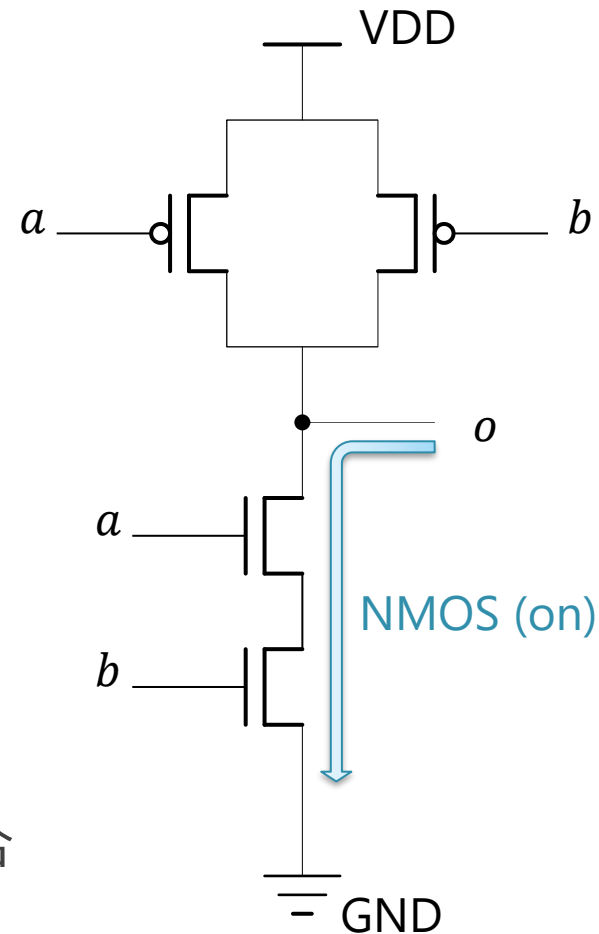
$$o = (a \cdot b)'$$

a	b	o
0	0	1
0	1	1
1	0	1
1	1	0

- a と b 共に 1（高電位）の場合

- PMOS（上側）は, off
- NMOS（下側）は, 双方 on = GND と繋がる

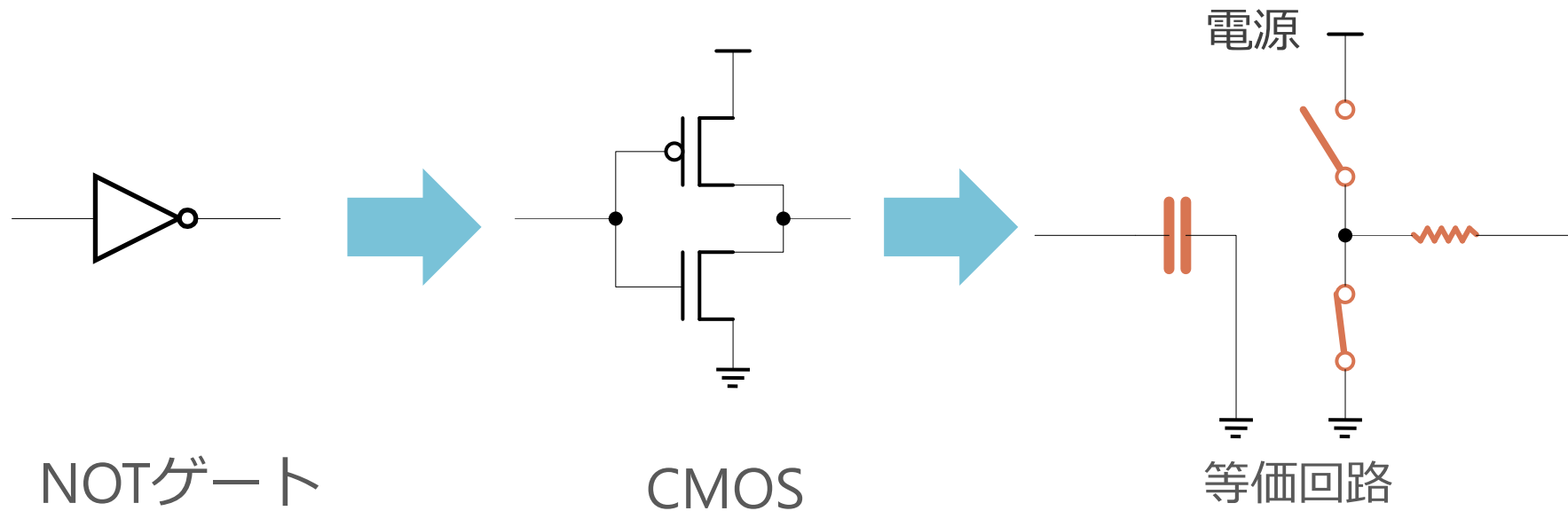
- 出力が 0（低電位に）



CMOS による論理回路の実現のまとめ

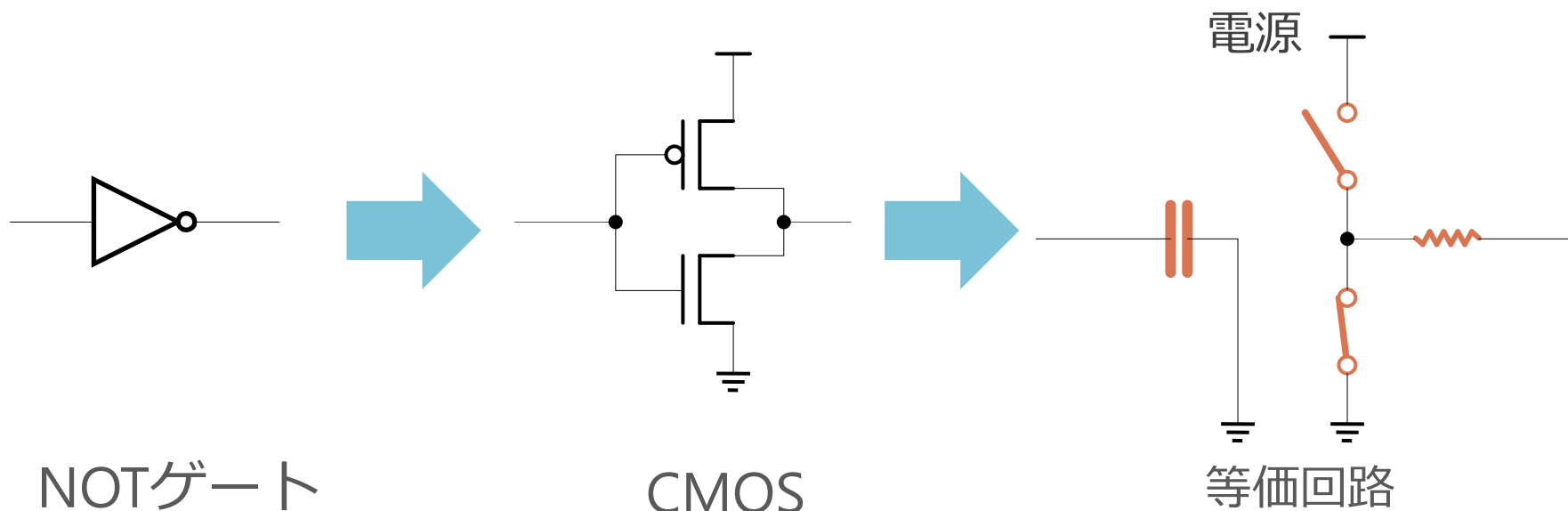
- 全ての論理ゲートは,
NMOS/PMOS の組み合わせによって構成されている
- リレーの時と同じ

CMOS ゲートの等価回路



- 抵抗 & コンデンサと，連動したスイッチによって表せる
 - コンデンサに充電：下のスイッチがON
 - コンデンサを放電：上のスイッチがON

CMOS ゲートの等価回路



■ コンデンサ :

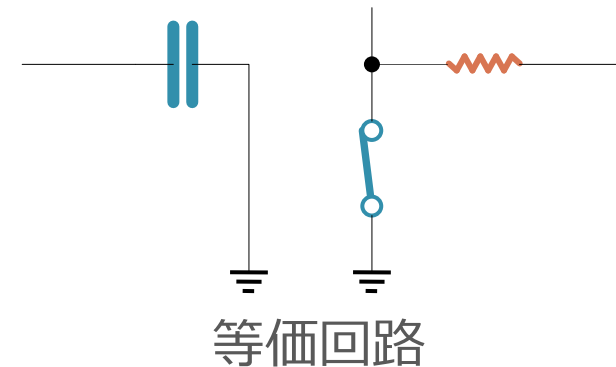
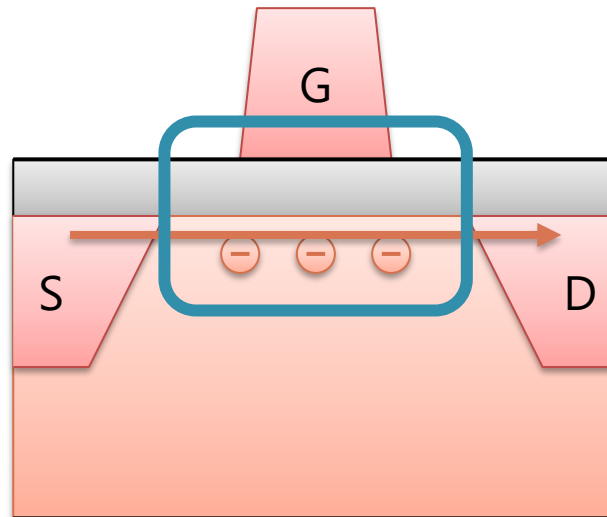
- 電荷を溜めることが出来るもの
- 電子を引き寄せてチャネルを作る = 充電している

■ 抵抗 :

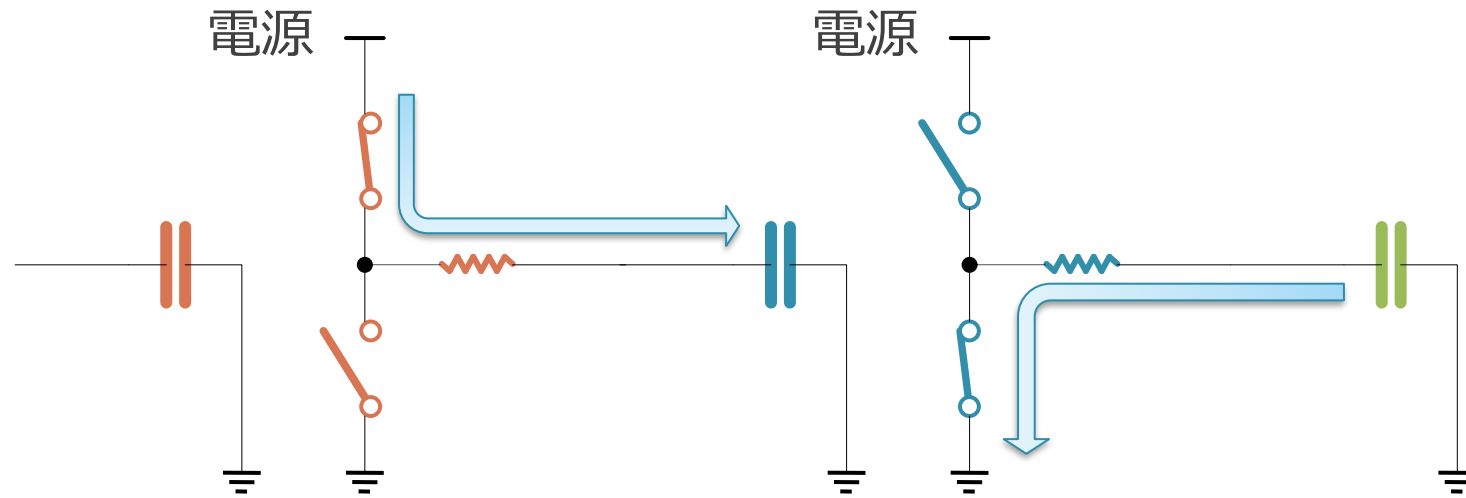
- 配線などの各部はちょっとずつ必ず抵抗がある
- 等価回路では, それを 1 つの抵抗として表現

CMOS ゲートの等価回路

- 青枠の部分がコンデンサとスイッチをなしている
 - 上下のサンドイッチ構造がコンデンサ
 - 左右に通すか通さないかがスイッチ
- 抵抗は全ての部分に存在する
 - 超伝導で無い限り，必ず抵抗が存在する



CMOS ゲートの遅延の実体



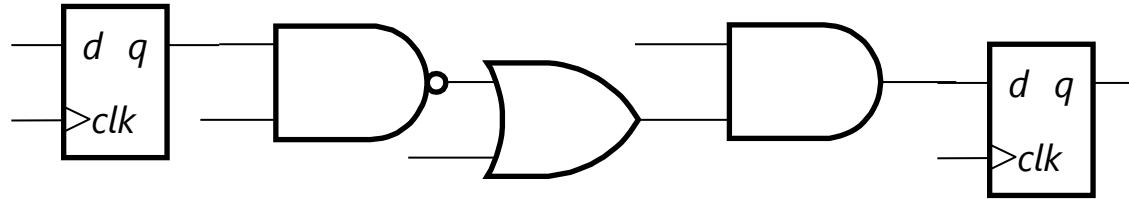
■ 遅延：コンデンサの充放電にかかる時間

1. あるゲートのスイッチが切り替わる
2. 次の段のゲートへの充放電が開始
3. 次の段のスイッチが切り替わる
4. ...

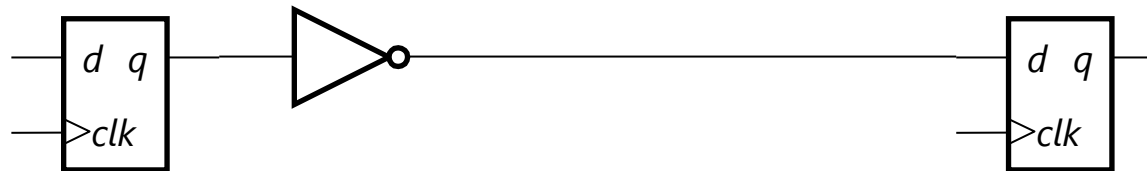
実際には

- 寄生容量：
 - トランジスタのゲート部分以外にも、あらゆる場所にコンデンサができてしまう
 - なにか導体が不導体をはさんで並べばコンデンサになる
- 寄生容量への充放電にも時間がとられる
 - 通常はトランジスタのゲートがメイン
 - 長い配線では、配線間にできてしまうコンデンサの影響も大きい

遅延の違い

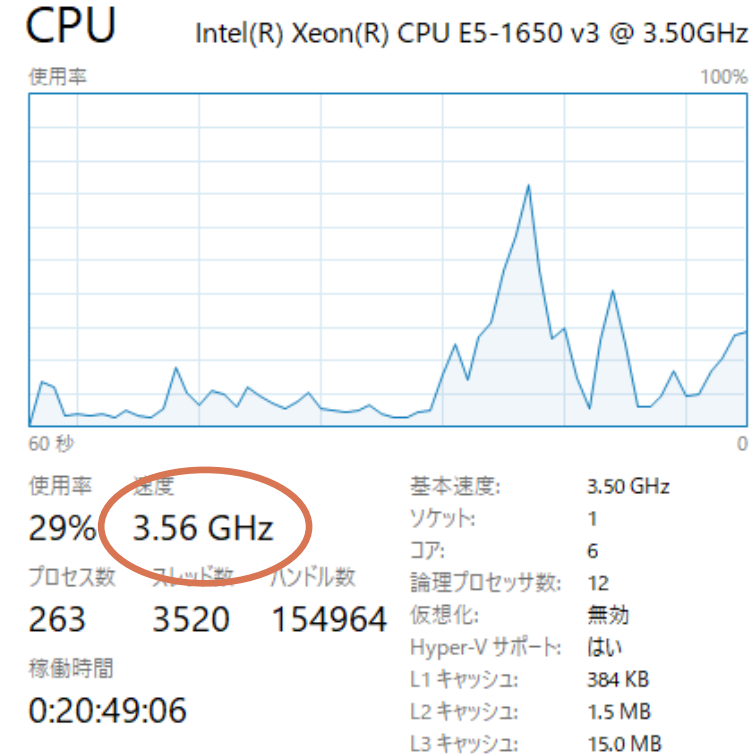
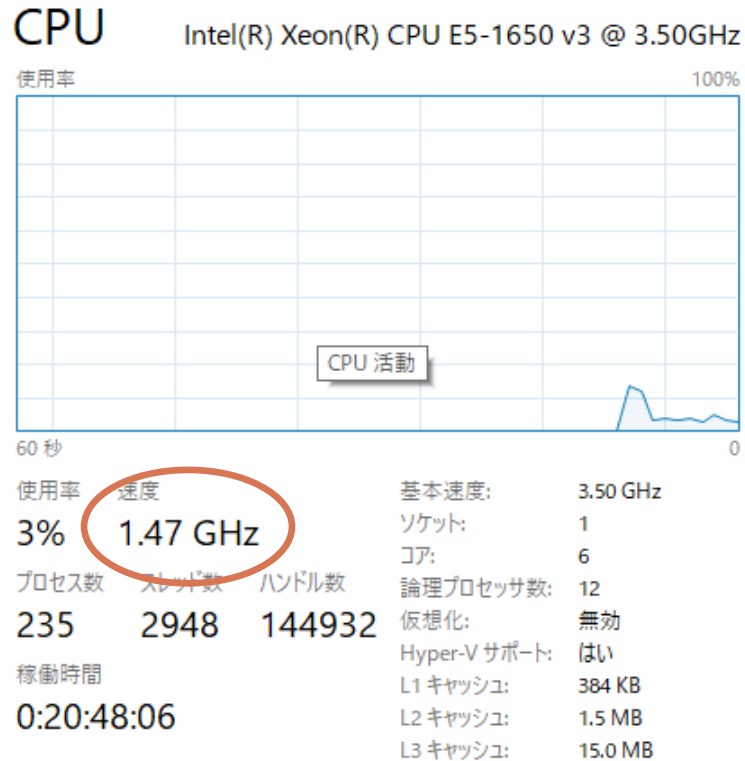


- 左側の D-FF から何個もゲートを経て右の D-FF に接続（遅い）
 - q が切り替わると,
 - 各ゲートのスイッチ（充放電）が順々に行われて,
 - 右側の d に伝わる



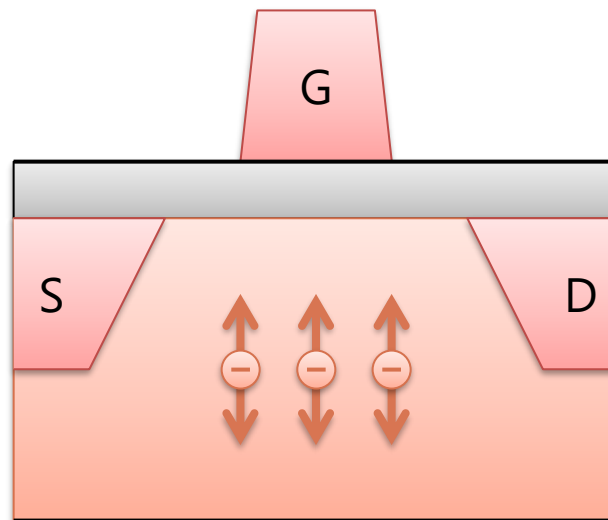
- 左側の D-FF から 1 つのゲートだけを経て右の D-FF に接続（速い）
 - ◇ q が切り替わると, 1 回だけスイッチ（充放電）が行われて,
 - ◇ 右側の d に伝わる

余談 : DVFS (Dynamic Voltage Frequency Scaling)



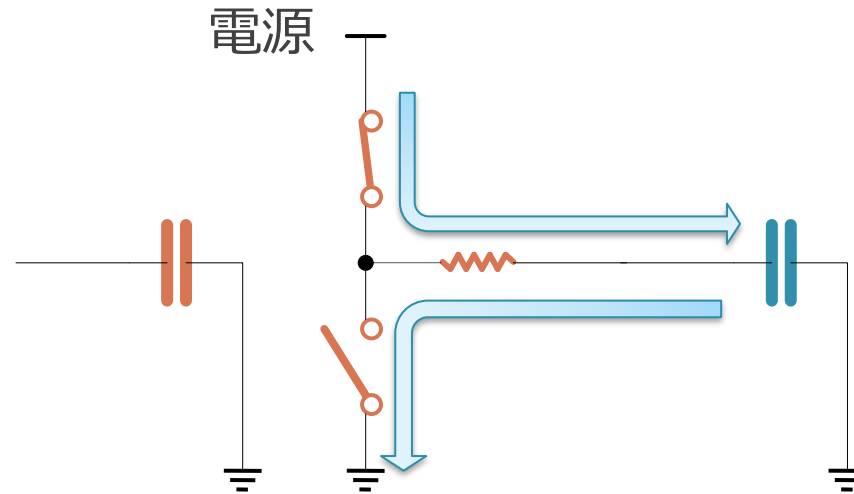
- CPU は通常, 負荷に応じて動作周波数を変えている
 - 消費電力削減のため
 - この時, 電圧も同時に操作している

電圧と遅延



- Gate にかける電圧を上げると、より高速に動作する
 - よりたくさん電子が上に集まる
 - チャンネルが厚くなり、電流が流れやすくなる
 - 次段のコンデンサの充放電が速くなる

消費エネルギー



- 消費エネルギーは、主にコンデンサへの充放電で消費される
 - 消費エネルギーは電圧の二乗に比例： $E = CV^2$
 - 電荷 $Q = CV$ が、電圧 V の分だけ電源から GND へ移動するから
 - 忘れた人は高校の物理の教科書を読もう
- 他にリーク電流と呼ばれるものによる消費もある
 - スイッチが一部ガバガバなので、常時多少漏れてる

DVFS: Dynamic Voltage Frequency Scaling

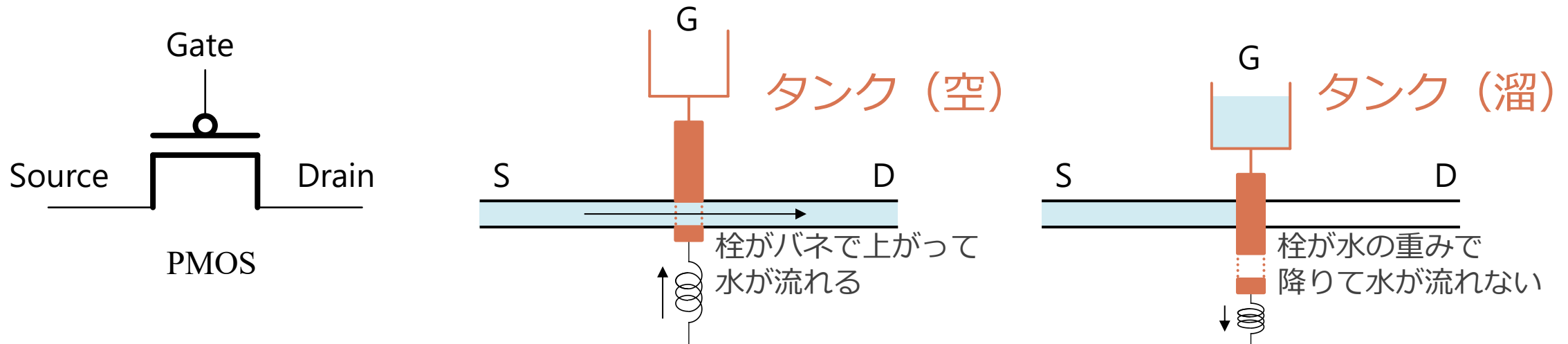
- 電圧と周波数の組を用意しておき、これを切り替える
 - 高速：高周波数（低遅延）で動かすために、電圧を上げる
 - 低速：低周波数（高遅延）でよいので、電圧を下げる
- 電圧を下げると、すごく消費電力が減る
 - 消費エネルギーは電圧の2乗に比例： $E = CV^2$
- 低周波数にすると充放電の回数自体（=周波数）も減る
 - 結果として、3乗のオーダーで電力が削減できる

水流システム

水流システム

- ここでは水の流れに例えて説明
 - 技術的に正確ではないが、かなりイメージは一致する
 - 一次近似としては良くあっているはず
 - 消費電力のイメージもかなり伝えやすい
- 回路の量（大きさ）と、電力の関係のイメージを掴んでほしい

水の流れて例えることにする



■ 水流タンクシステム：

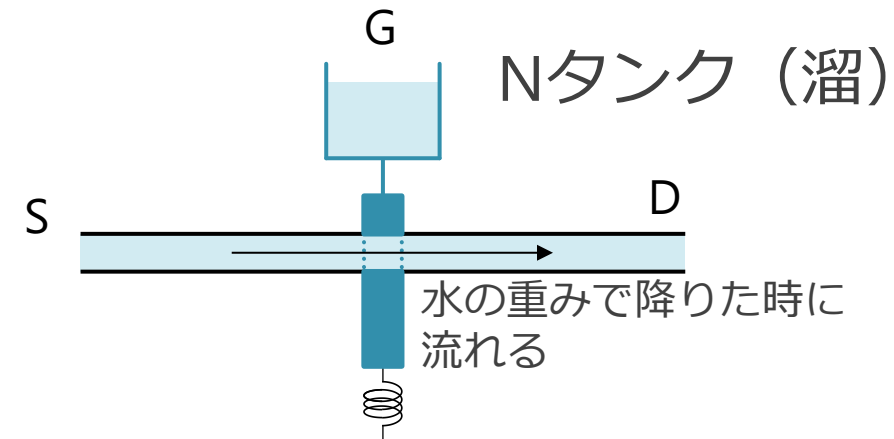
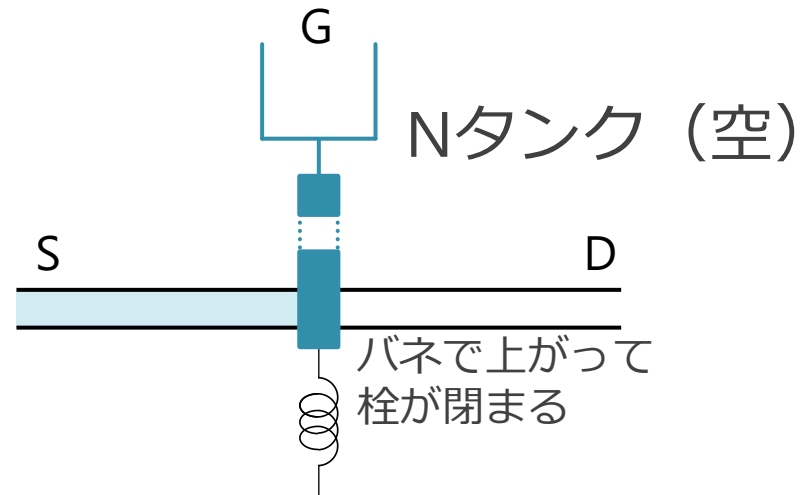
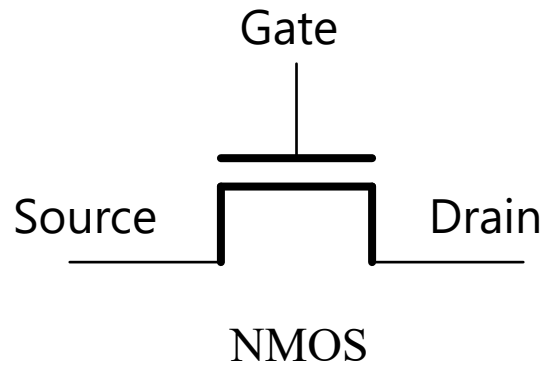
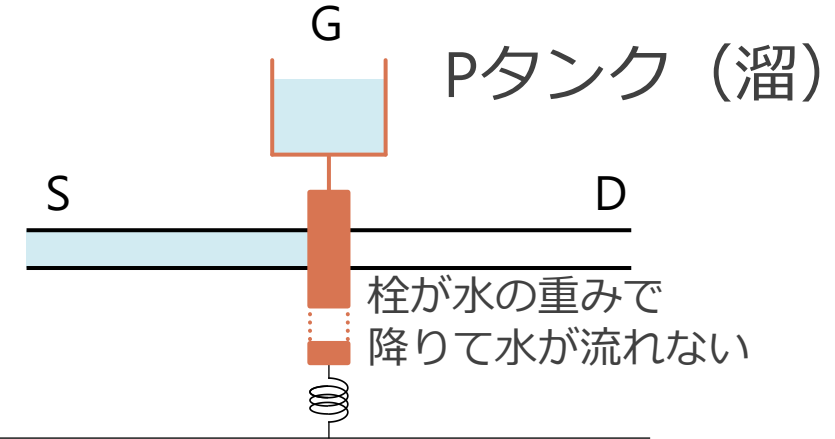
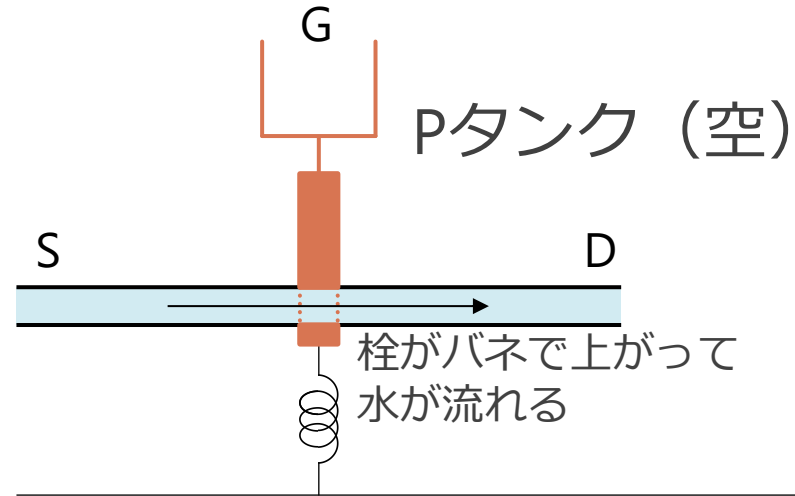
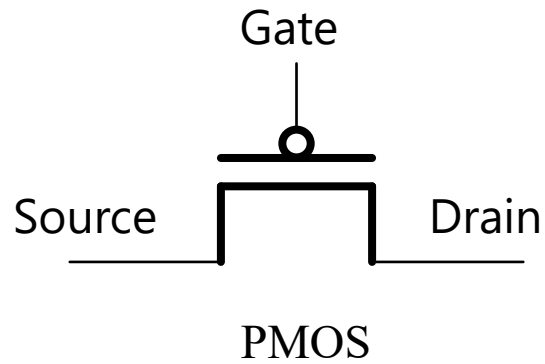
- 水が流れるパイプとタンク+栓を考える
- タンクに水がたまっているかどうかで連動して動く栓を持つ
 - 栓には水が通れる部分があって、水流を制御できる

■ PMOS と同様の挙動：

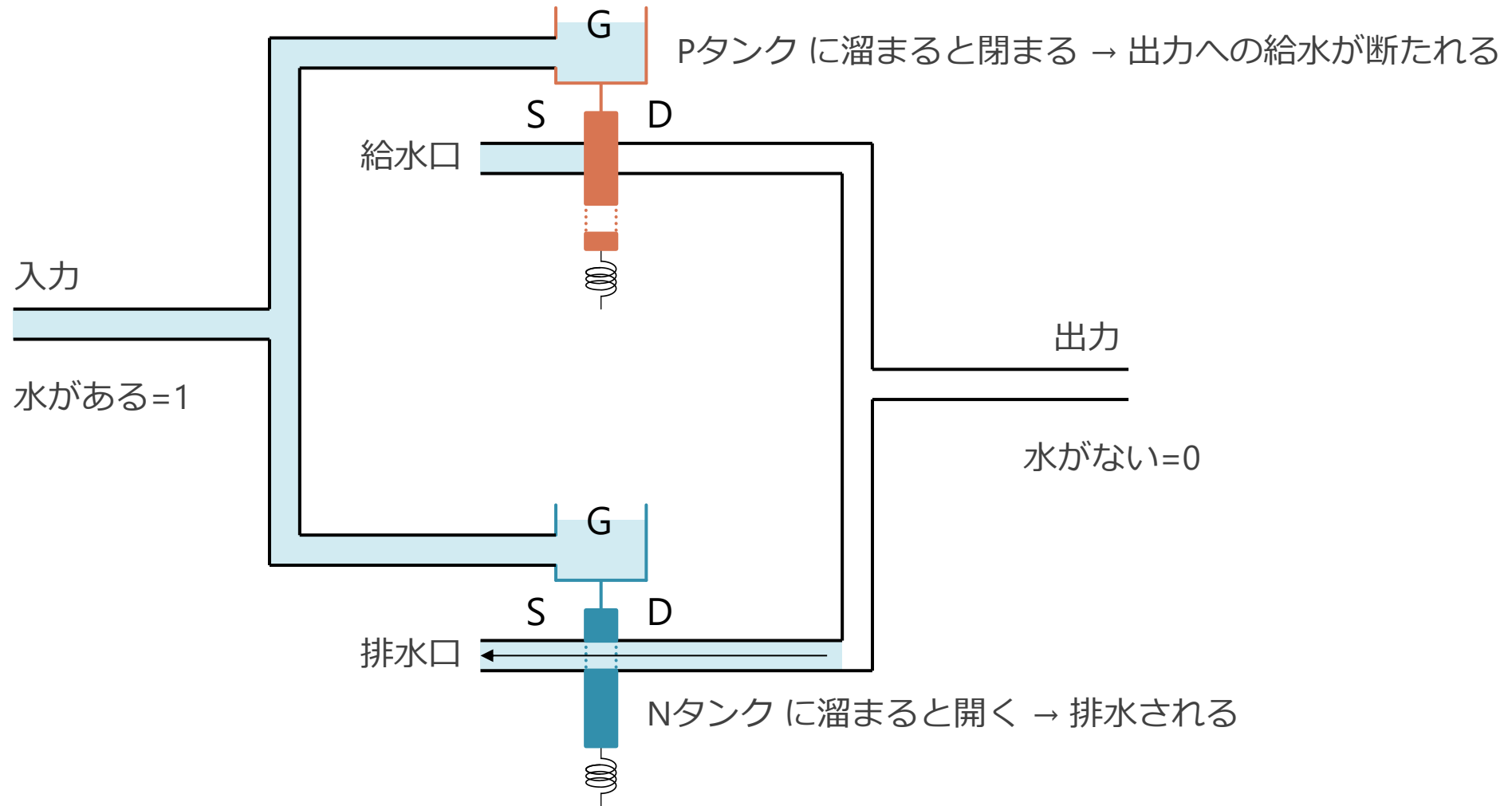
- 「タンクが空だと、S から D に水が流れる」
- 「電荷が Gate から抜けて空だと、S から D に電荷が流れる」

タンクは2種類ある (PMOS と NMOS に相当)

タンクに水が溜まっている時に、流れるか遮断されるかが逆になっている

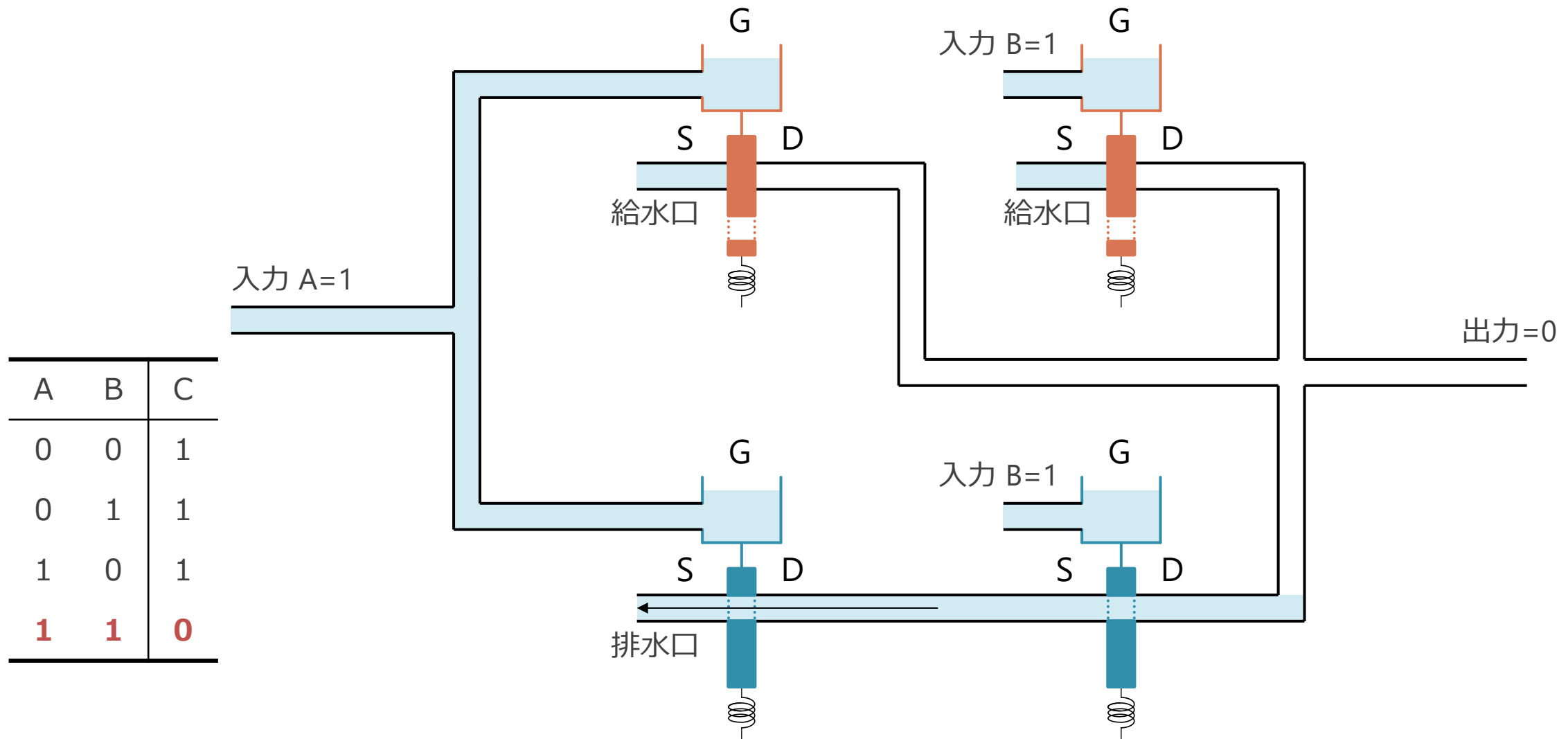


P タンクと Nタンクで NOT ゲートが作れる



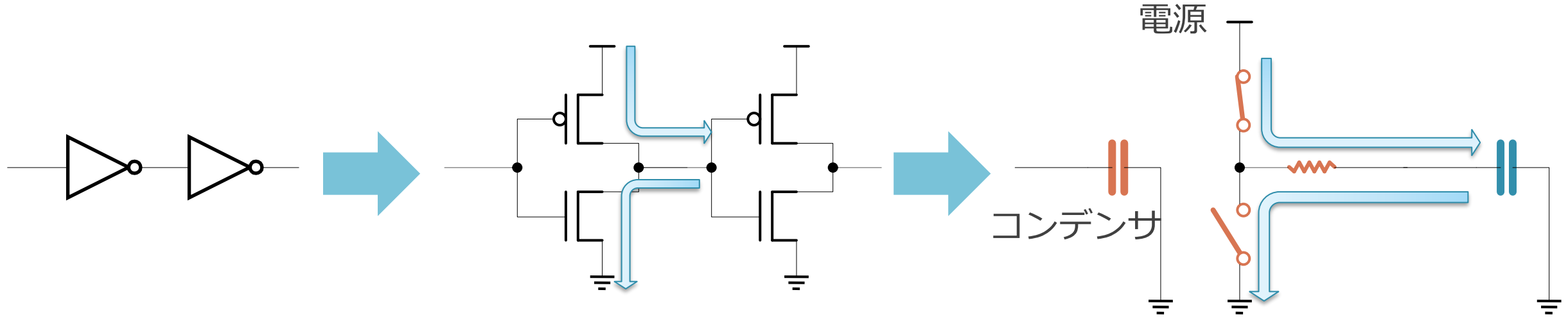
- 給水口には常に一定の圧力で水が供給されている (=電源)
- 入力側の水の有無により, 出力側の水の有無が反転 → NOT ゲートの動作

上側を並列繋ぎ、下側を直列繋ぎにすると NAND になる



- 下側の2つのNタンクは、直列に繋がってるので双方溜まった時のみ排水が有効 → NAND

CMOS 回路におけるスイッチングの消費エネルギー

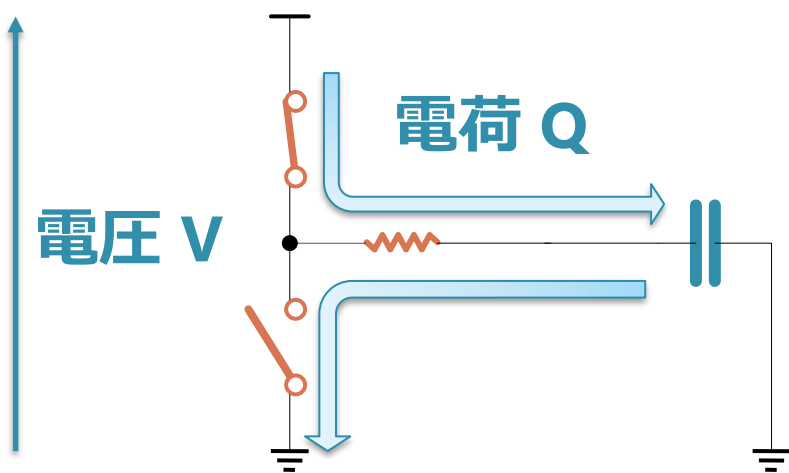


- CMOS 回路は、コンデンサに連動したスイッチとみなせる
(MOS 構造のゲートのところがコンデンサになってる)
 - コンデンサが充電状態：下のスイッチがON
 - コンデンサが放電状態：上のスイッチがON
- 主に計算で消費されるエネルギー：
 - スイッチを ON/OFF するためのコンデンサへの充放電で消費

これも水流で考える

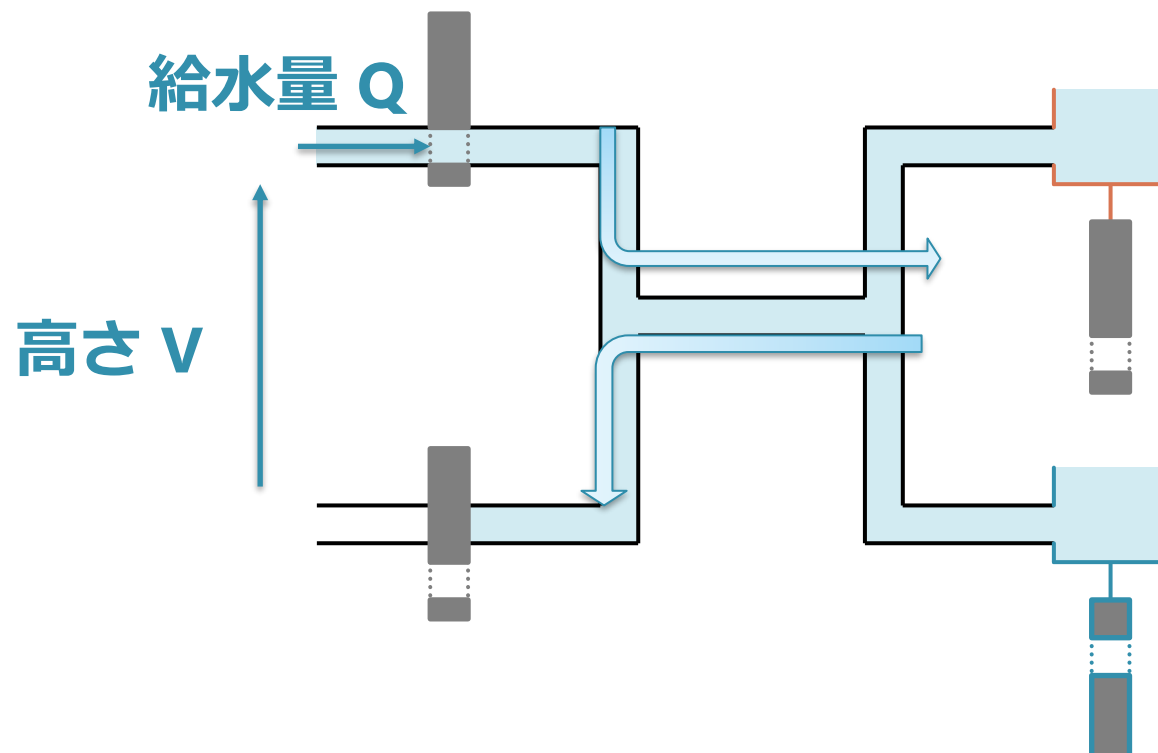
■ 電流を流して失われるエネルギー

- コンデンサに溜まる
電荷の量 : Q
- 電圧 : V
- 消費エネルギー : $Q \times V$

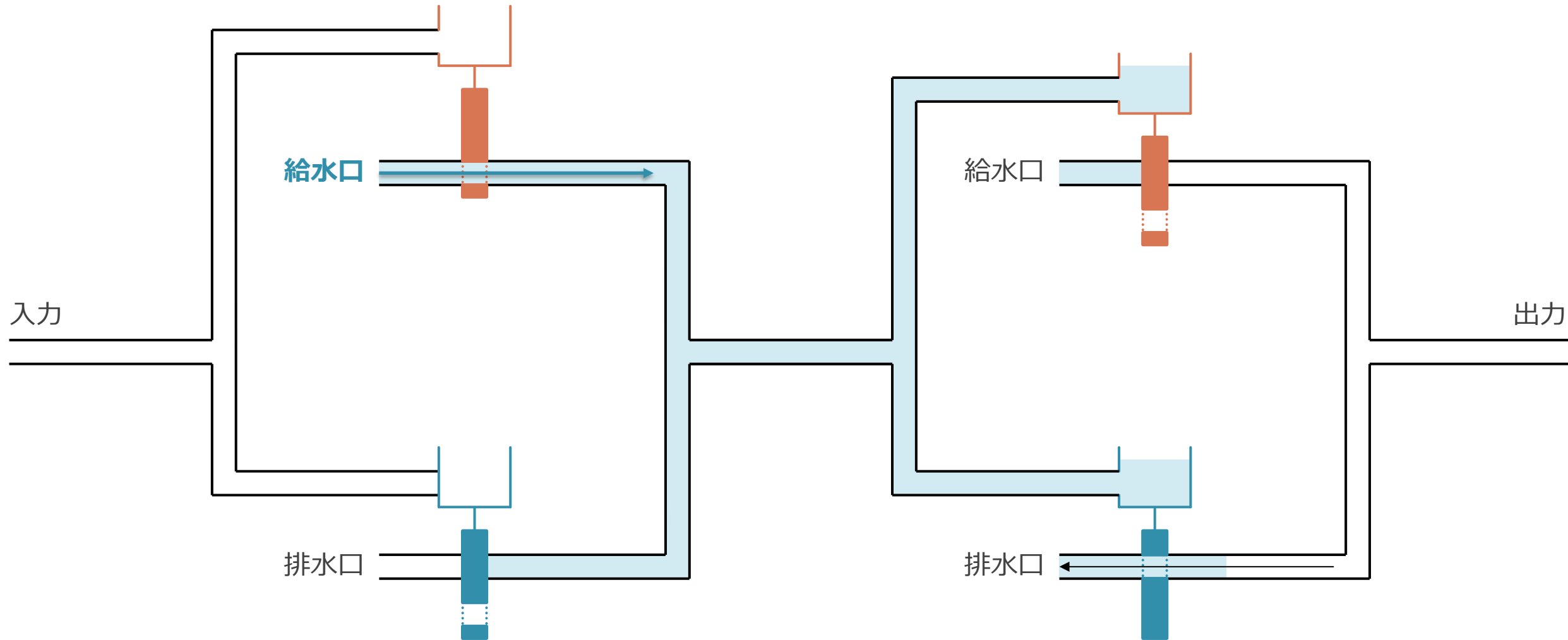


■ 水を高いところから流すとその分位置エネルギーが失われる

- 水の総量 : Q
- 排水口から給水口高さ : V
- エネルギー : $Q \times V$

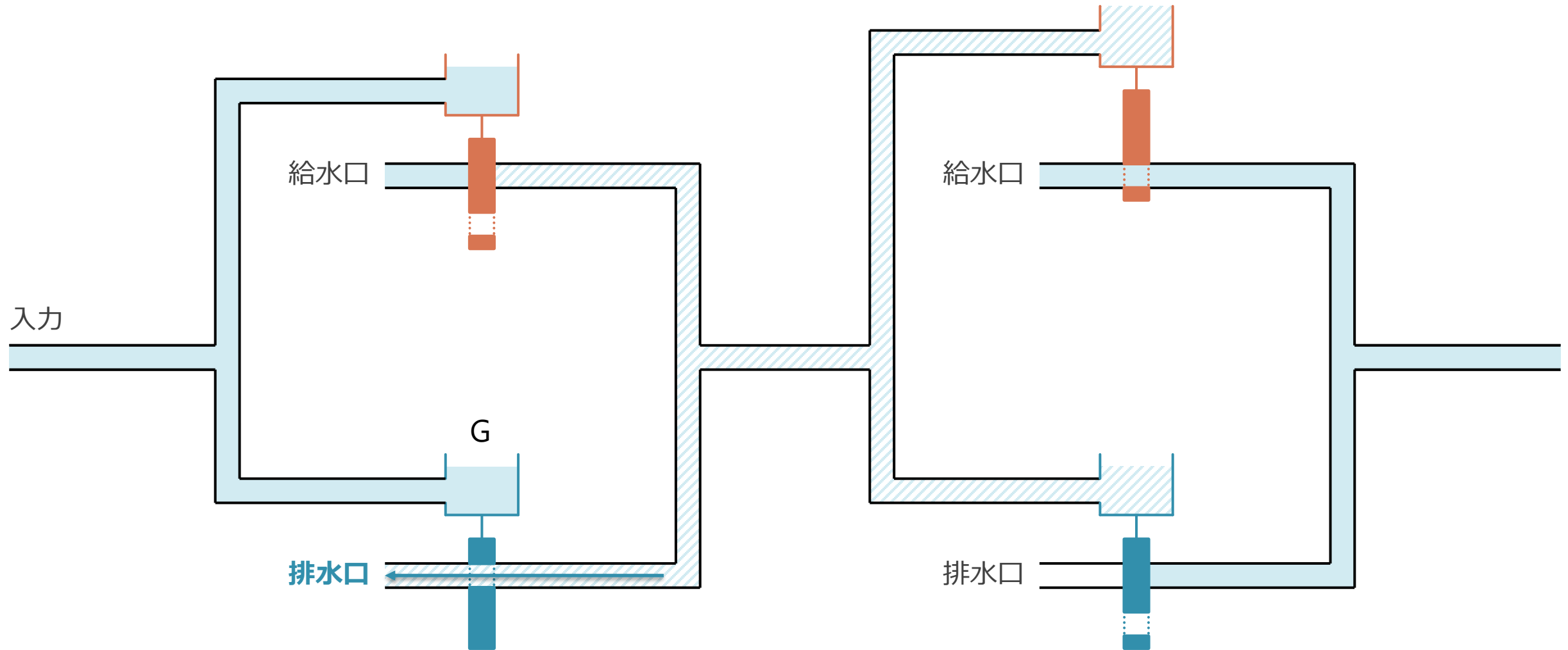


NOT ゲートを 2 段重ねた例：入力が 0 の場合



- 左上の給水口から，右側 2 つのタンクに給水される

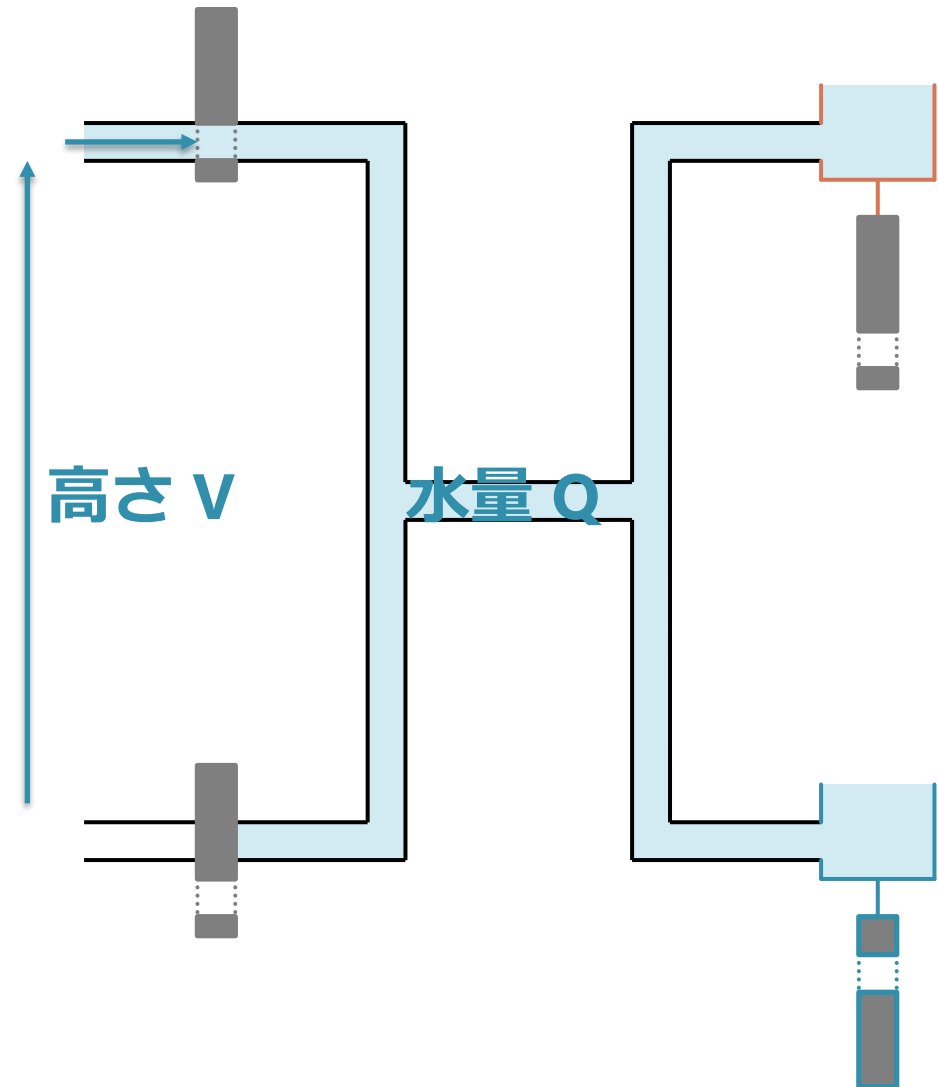
入力が 0 → 1 に遷移すると、
中央と右側タンクの水（斜線部分）が排水される



- 入力が水がくると、右側の2つのタンクにあった水が排水される

つまり、スイッチの切り替えが起きると・・・

- スイッチングが起きると・・・
 - 高さ V の給水源から水量 Q が供給
 - 総量 Q の水がタンクとパイプに入って抜ける
- 入力の変化に伴い、水の出し入れが生じるとエネルギーが消費される
 - 電荷が電源から GND に流れるのも同じ
- 水を高いところから流すとその分位置エネルギーが失われる
 - タンクにたまった水の総量： Q
 - 排水口から給水口高さ： V
 - エネルギー： $Q \times V$



補足：スイッチの ON/OFF 以外による消費エネルギー

- その他に、リーク電流と呼ばれるものもある
 - トランジスタを OFF にしていても、流れ続けてしまう電流
- 分類：
 - 充放電によるもの：動的（dynamic）消費電力
 - リークによるもの：静的（static）消費電力
- リークは、栓が完璧ではないのでチロチロ水が漏れているイメージ

水流システムのアナロジーから説明できること

■ 消費エネルギー：

- 回路：全 PMOS/NMOS のコンデンサに溜まる電荷量 Q と、電圧 V によって決まる
- 水流：全タンクにたまる総水量 Q と、高さ V によって決まる

■ 回路量に比例して消費エネルギーは増える：

- 水流：全タンクに溜まる総水量はタンクの数に比例する

■ 配線も電力を消費する：

- 回路：配線が持つ寄生容量に充放電が起きる
- 水流：パイプにも水は多少溜まる

■ 半導体化が微細化すると電力や速度も向上する：

- 回路：コンデンサに電荷をためる量と時間が減る
- 水流：タンクが小さくなるので、水が貯まる時間と量が減る

まとめ：一番下から積み上げて一番上までいけた

一番下がすぐ変わっても、上は同様に作れる

アプリケーション・ソフトウェア
画像処理 / 音声認識 / 言語処理 / 機械制御
機械学習 / AI / WEB サービス / 暗号 ...

システム・ソフトウェア
OS / コンパイラ / インタプリタ

(狭義の) コンピュータ・アーキテクチャ

論理回路

CMOS/リレー/真空管/レッドストーン etc...

リレーや真空管と比較した CMOS の利点

- 超小さい：CMOS はナノメートル単位ぐらいの大きさで作れる
 - リレーや真空管は頑張っても数 cm ぐらい
- 超速い：CMOS はナノ秒以下で ON/OFF できる
 - リレーは物理的に動くので、ミリ秒ぐらいはかかる
 - リレーよりマシだが電子の移動距離が長いので遅い

リレーや真空管と比較した CMOS の利点 2

■ 超省電力

- リレーは物理的に動くので結構電気を食う
- 真空管は常に暖め続けるのでめっちゃ電気を食う

■ 壊れない：

- リレーは物理的に動くので、バネとか劣化するし、かなり壊れる
- 真空管はフィラメントがそのうち焼き切れるし、ガラスが割れる

まとめ

■ 論理回路の実現方法

- リレー
- CMOS

■ 論理回路の遅延と消費エネルギー

- CMOS の場合, 充放電にかかる時間とエネルギー

課題 4

- 1 0 から 3 まで 1 ずつ下りながら数える以下のループをアセンブリ言語で書け
 - 使用する命令セットは第 2 回の講義資料のものに準じる
- ヒント :
 - 減算には add の代わりに sub を使う
 - 初期値は li 命令で設定
 - 講義中の for 文の例のようにメモリ上に i を置くとややこしいので, レジスタの上で全て終わらした方が楽

```
1: for (i = 10; i > 3; i-=1) {  
2: }
```

提出方法

- 以下の2つを提出（双方が必須であり，成績に関係します）：
 - 課題の提出は Moodle の今日の日付のところからお願いします
 - 感想や質問を「感想や質問」のところに投稿してください
 - わからない場所がある場合，具体的に書いてもらえると良いです
- 提出締め切り
 - 次回講義開始まで
- 注意：
 - 課題の出来は，ある程度努力したあとがあれば良しです
 - 必ずしも正解していなくても良いです

実際の回路生成

```
if code == 100:  
    out = rs1 xor rs2  
elif code == 110:  
    out = rs1 or rs2  
else:  
    out = rs1 and rs2
```

1. ハードウェア記述言語から，論理関数を生成：
 - ナイーブには，入力の全パターンに対して出力を記録した真理値表
 - 実際には表のサイズが爆発して，真理値表ではすぐに破綻
 - 表のサイズ = 2 の入力ビット数乗
 - さまざまな効率的な保持方法が提案されており，使用されている
 - Binary Decision Diagram (BDD)

実際の回路生成

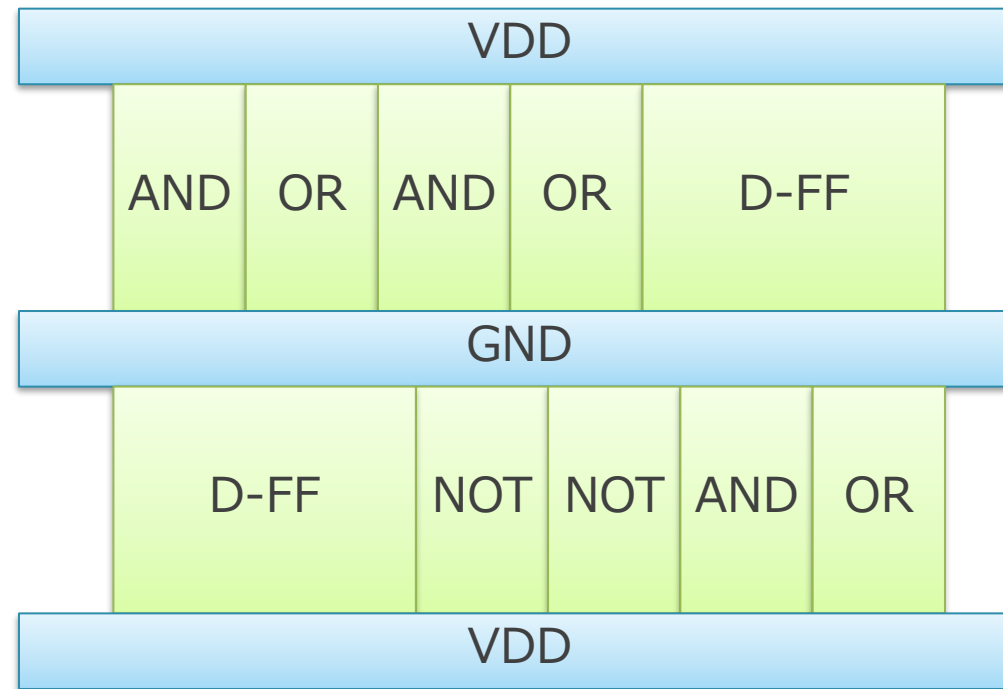
1. 論理合成：

1. ハードウェア記述言語から，論理関数を生成
2. 生成された論理関数を最適化
 - カルノー図：4入力ぐらいが限界
 - クワイン・マクラスキー法：入力数が増えると計算量が爆発
 - さまざまな最適化アルゴリズムが提案・使用されている
3. 回路のプリミティブと接続関係を生成
 - AND, OR, NOT, D-FF を始めとして，よく現れる回路の部品が用意される

2. 配置・配線：

1. 回路のプリミティブを配置・配線

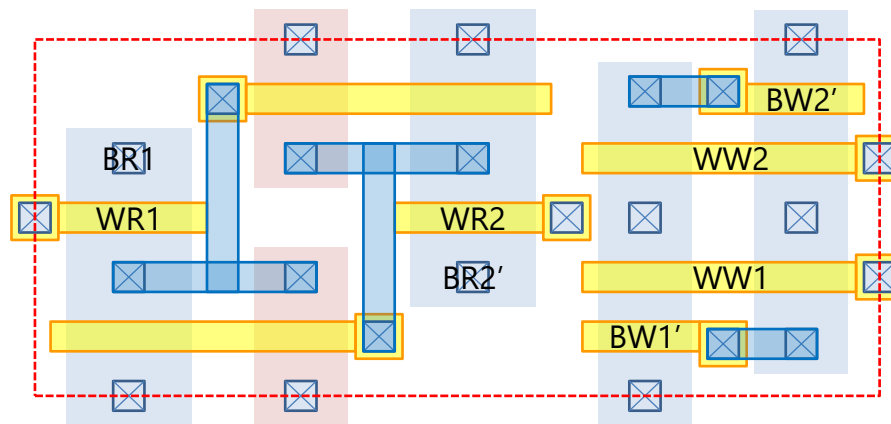
スタンダード・セル



■ スタンダード・セル：

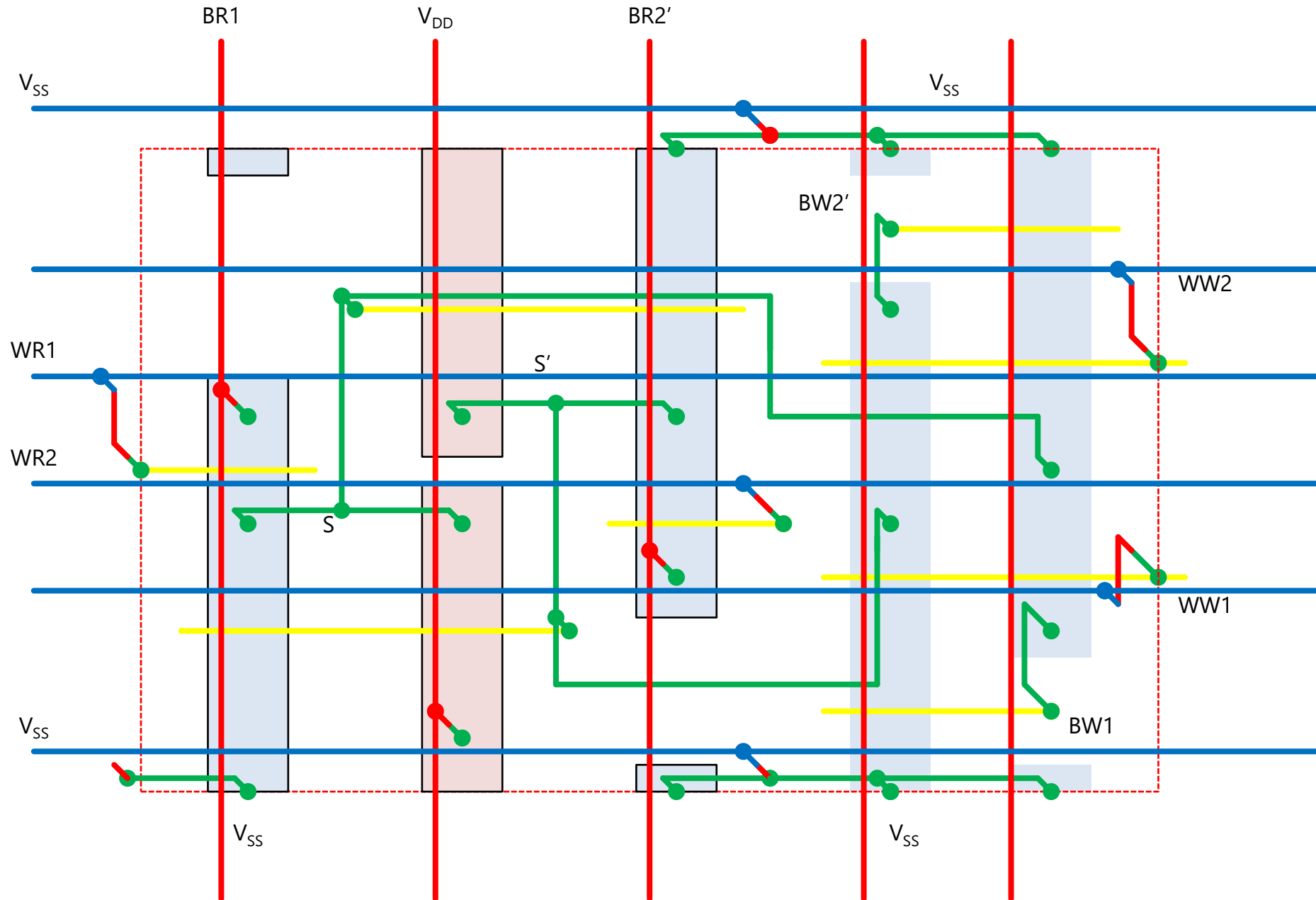
- AND や OR のようなプリミティブとなる回路「セル」を用意
 - 高さを幅を揃えておいて，簡単に並べられるように作ってある
- これらを配置して，配線を接続することにより回路を実現
 - 配置配線という

フルカスタム・レイアウト

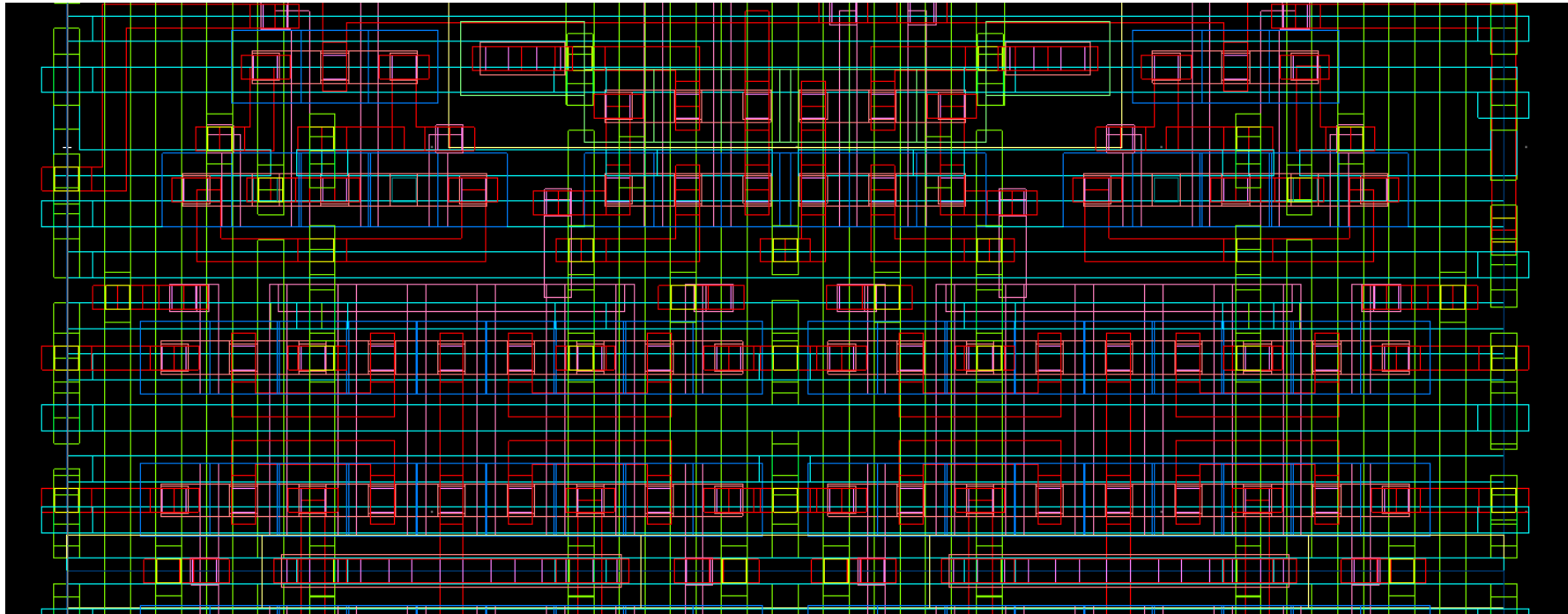


- 各スタンダード・セル（ゲート）は複数のトランジスタにより構成
- フルカスタム・レイアウト：
 - 半導体チップ上に1つ1つトランジスタをお絵かきして設計
 - 形状が特定のパターンを満たすと，トランジスタとして機能
 - 尋常じゃない手間がかかる
- 今はスタセルの中身と，本当に性能が出したい部分だけこれで作る
 - ごく一部だけアセンブリ言語でプログラムするようなもの

3次元的な構造を考えながら，設計

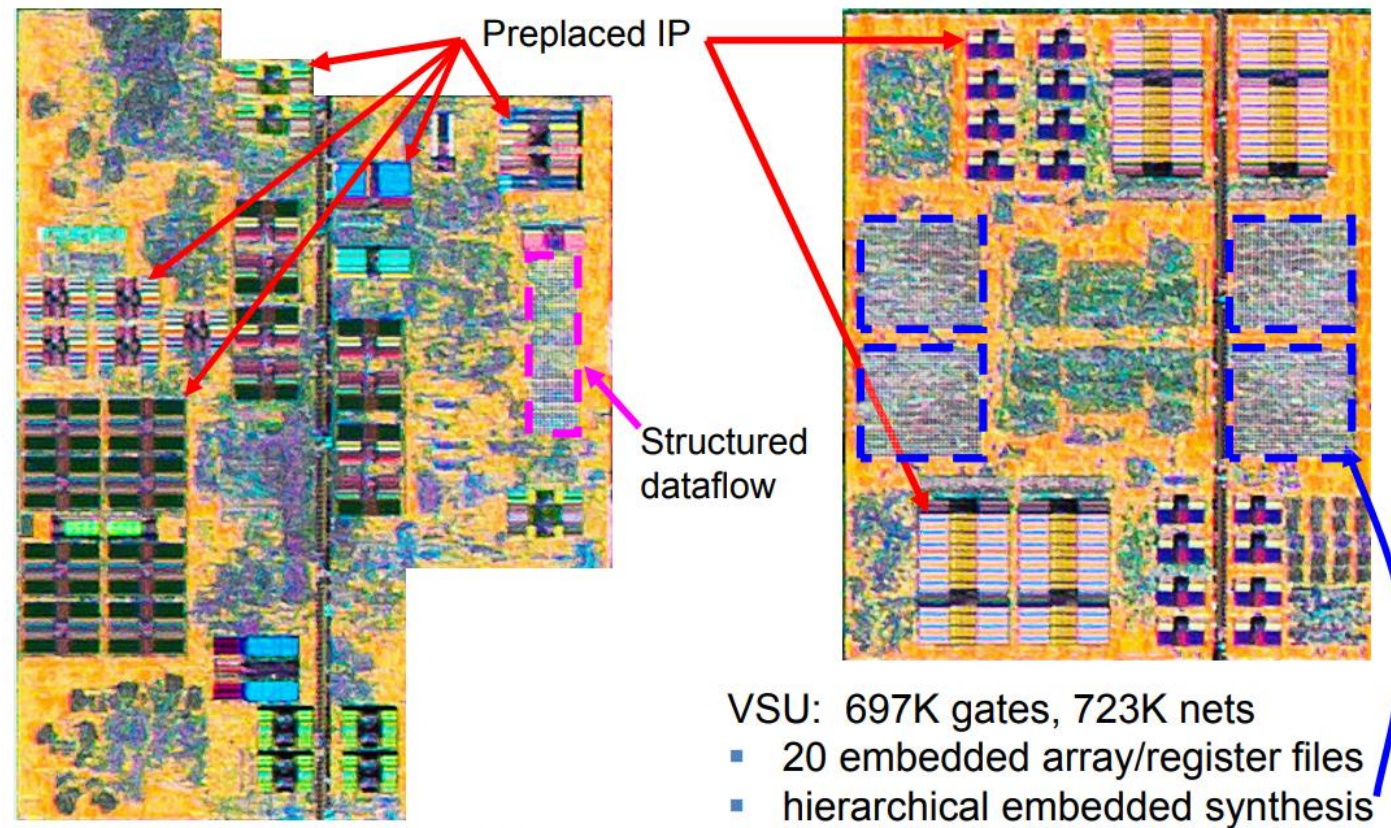


実際の設計データ



実際の例：IBM POWER8 のチップ写真

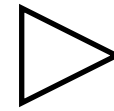
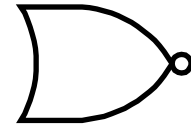
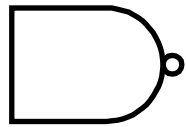
画像は 2014 IEEE International Solid-State Circuits Conference 5.1: POWER8™: A 12-Core Server-Class Processor in 22nm SOI with 7.6Tb/s Off-Chip Bandwidth より



IFU: 580K gates, 628K nets

- 37 embedded array/register files

- もわっとした模様の部分がスタンダード・セルを自動で配置した部分
- 四角いところは基本的にはメモリ
 - 同じものが規則正しく並んでいる



感想や質問

- 課題では 0b10 (2) での割り算を行いましたが、もし割る数が 0b11 (3) のように、1ビットシフトだけでは計算できない中途半端な数だったり、割り切れない・余りが出る場合は、コンピュータの内部ではどのように処理されているのでしょうか？

- p.63のD-EFの実現例の理解に苦しみます。上の図にデータ入力繋がっていませんが、これはどういう状態ですか。
- 私が聞き逃してしまったのですが、D-FFの実現例のページ(p63)の図がいまいちよく分かってません。下の図でdの値が入力されて、上の図でdの値が出力されるという認識で当たってますか？

- 論理ゲートをループさせることで記憶を生む、という方法が印象的でした。なんとなく信号が後ろに回り込むことで過去の入力が今回の出力に影響を与えるというようなイメージで理解しましたが、厳密になぜそうなるかを考えるのは難しかったです。

- すみません。特に感想が思いつきませんでした。

- 16進数の具体的な使いどころがよく分かりませんでした。
プログラムを人間側が読む時にわかりやすいよう、という認識で正しいですか。
具体例があったらお聞きしたいです。

- 16進数→2進数の時に、求めた2進数が0b0～1～の形になった時は数部分の頭の0を省略して0b1～の形式にした方がいいですか？

- 高校の授業で「10進数の0.1などは2進数で正確に表せない」ことに「そんなに雑でいいのか!？」とモヤモヤした記憶があるので、BCDという仕組みが存在していることに安心しました。

- D-ff、保存しつつ反転も戻しつつ、となっていて、よくできているなあと思いました。回路としては単純ですが私には絶対思いつかないなあと思ったのですが、もともともっと複雑だったものを次の人が改良して...みたいに多くの人に関わっていたのでしょうか。一人だとしたら驚きです。

- 授業中サラッとおっしゃってましたが、テストは何を持ち込むことができるのでしょうか？もう一度言ってもらえるとありがたいです。



- スマートフォンなど記憶したい容量が多い場合は、4進数や16進数を用いることもあるとのことですが、回路は2進数と比べてどれくらい複雑になりますか？
- また、4進数と16進数では複雑さがかなり違うように感じました。この部分は16進数、などの具体例はありますか？

- マルチプレクサは複数の入力から、1つを選択して出力するというこ
とで、その選択はクロックの立ち上がりのたびに行われるという認識
で間違いありませんか？また、その選択はどのように行われるのです
か？

- 論理回路の完全生の説明がありましたが、 $\{\text{AND}, \text{NOT}\}$ で完全集合だということは、効率や見やすさを無視すれば、実質ORは不要だということですか？

- D-FFの説明で、データ入力が d でデータ出力が q なのはなぜですか？

- 一個前の課題の質問なのですが、命令セットにld命令やst命令が入っていない場合、プログラムをアセンブリ言語で表す時これらの命令は書かないということですか？

- 質問ですが、0bなどの後にスペースは必要ですか？
- 課題で2進数を表記する際、先頭の0を省略するかどうかで少し迷いました。実際のプログラミングやハードウェアの設計を考える場合は、桁数を一定に揃えるために先頭の0を残す表記と、数学的に0を省略する表記とでは、どちらが一般的によく使われるのでしょうか？

- D-FFの動作がシフト演算に似ていました。クロックの立ち上がりのたびに2で割ってあるように見えるので、 2^2 や 2^4 などで割って早送りやスキップができれば面白そうだと思いました。

- 課題2の解答でsubではなくaddみたいな訂正があったと思うのですが聞き逃してしまったのでもう一度教えて頂きたいです。

解答例 (b)

(b)

```
1: i=4;  
2: if (i <= 1)  
3:   i--;  
4: i++;
```

(1)

```
li A←4  
b A>1, LABEL  
sub A←A-1  
LABEL:  
add A←A+1
```

(2)

```
li A←4  
b A>1, LABEL  
add A←A+1 << ここが sub になってた
```

- 演算やc言語でいうif文のアセンブリ言語が講義で出てきましたがfor文や構造体のアセンブリ言語は用意されていますか。

- 16進数について、今までなんでそんなに中途半端な数にしたんだと思っていたけれど、16が2の累乗であるだけでなく2の4乗であり、この4ビットで一桁が表せるという特性も2の累乗であることで、非常に都合の良い数字だということを知って理にかなっているのだと感じスッキリした。

- 16進数の英字は、大文字小文字に特に意味はないのでしょうか。

質問や感想など

- 論理回路のパートに入ってからついていけなくなっていました。、論理回路は、何をするための論理回路なのかが全く分かっていませんでした。
- 調べたところ、判断、計算、記憶、制御というコンピュータの主な働きを全て実現するということが分かったのですが、それがわかってもなお、講義資料を見ても全く意味が入ってこないなので勉強が足りないと思いました。論理回路は完璧に理解するべきでしょうか。

- 論理回路の動きとしてこれまた 2 進数が基盤となることがわかったが、実際に 2 進数よりも 16 進数に優先的に変換される場面はどのようなときか気になった。

- 情報という物理的実体のないものを扱う論理回路において、なぜ遅延や消費電力が発生するのかという疑問が、コンデンサの充放電という物理現象の観点から理解できました。また、リレーやCMOSによるNANDゲートの構築からCPUの部品、そしてコンピュータの完成へと、ボトムアップに積み上がっていく構造から、基本素子からプログラム実行に至るまでの繋がりを実感しました。

- $\{\text{AND}, \text{OR}, \text{NOT}\}$ 、 $\{\text{AND}, \text{NOT}\}$ 、 $\{\text{OR}, \text{NOT}\}$ が完全集合の例であることはわかるが、 $\{\text{NAND}\}$ や $\{\text{NOR}\}$ だけでも完全集合になることがすぐには納得できなくて驚いた。

- 私はアセンブリ言語よりもC言語の方がわかりやすく感じました。アセンブリ言語はコンピュータに近い低水準な言語であり、命令一つ一つの意味を考えながら読む必要があるため難しかったです。一方で、C言語は処理の流れを直感的に理解しやすく、これまで学んできた内容ともつながって理解しやすいと感じました。